

Key Updatable Hash Based VRF

Suman Ghosh, Ratna Dutta and Sourav Mukhopadhyay

Department of Mathematics, Indian Institute of Technology Kharagpur
Kharagpur, 721302, India
`{sg7@kgpian, ratna@maths, sourav@maths}.iitkgp.ac.in`

Abstract. Unbiased, unpredictable, and publicly verifiable randomness is essential for a wide range of blockchain-based Web3 applications. Verifiable Random Functions (VRFs) naturally satisfy these requirements. For practical deployment, however, a VRF scheme must support efficient key generation and allow multiple evaluations across different blockchain rounds. In this work, we present a post-quantum secure, key-updatable VRF construction built from symmetric cryptographic primitives, including hash functions and pseudorandom generators (PRGs). The core of our design is a quantum-secure Extended Merkle Signature Scheme (XMSS) structured over multiple layers. We reorganize the XMSS framework in a systematic way to integrate it seamlessly into our VRF construction. Compared to existing approaches, our scheme offers improved key generation efficiency while enabling multiple evaluations from a single secret-verification key pair.

Keywords: Post-Quantum · Verifiable Random Function · Hash · Merkle Tree · Blockchain · Algorand.

1 Introduction

A Verifiable Random Function (VRF) allows the generation of authenticated randomness that can be efficiently and publicly verified. This was put forth by Micali, Rabin and Vadhan [41] in 1999. VRFs are utilized as a fundamental component in numerous applications including Domain Name System Security Extensions (DNSSEC) protocol [27], Web3-based applications such as decentralized finance and GameFi [43] and in blockchain consensus protocols [17, 25, 16] to establish Proof-of-Stake to supply verifiable random inputs essential to the protocols.

VRF operates through four main algorithms. First, public parameters are generated using algorithm $\text{ParamGen}(1^\lambda)$, which sets up the public parameter pp based on a security parameter λ . A user can run algorithm $\text{KeyGen}(\text{pp})$ to generate a key pair comprising a secret key sk that is kept private to the user and a verification key vk that is published. Using sk , the user executes algorithm $\text{Eval}(\text{pp}, \text{sk}, x)$ on an input x to obtain a pseudorandom output y and a proof π showing that y is correctly computed. Finally, verifier calls algorithm $\text{Verify}(\text{pp}, \text{vk}, x, y, \pi)$ to check the validity of the output y and the proof π using the verification key vk , pp and input x . If verification succeeds, y is accepted as

the correct VRF output; otherwise, it is rejected. A secure VRF should satisfy two key properties - *uniqueness* and *pseudorandomness*. Uniqueness means that for any given input and verification key, there exists only one valid VRF output that can be accepted by the verification algorithm. It ensures that no one can produce two different valid outputs for the same input. Pseudorandomness means that the VRF output appears completely random to anyone who does not possess the secret key. In other words, without the secret key, the output is indistinguishable from a truly random value.

BLS-VRF [9, 8] and ECVRF [42] are the most common VRF constructions which rely on cryptographic pairings and elliptic curves respectively and are vulnerable to adversaries equipped with quantum computers. In blockchain systems such as Algorand [25], VRFs are employed to guarantee the honest selection of committee members for all finalized blocks. Each user independently runs a VRF using their secret key and the current round information as input; if the VRF output is below a threshold determined by their stake, the user is selected as a committee member for that round. A newly joining participant, even without access to prior block history, can ascertain the validity of the chain by examining the recorded votes, assuming the continued security of the VRF. In such protocols, since each block is approved by a majority of committee members, forbidding blockchain forks. A blockchain fork occurs when the chain splits into two due to conflicting blocks or protocol changes. It's undesirable because it can cause inconsistencies, double-spending risks, and loss of consensus among network participants. However, if the VRF's security is compromised, an adversary could manipulate the selection of committee members for any round (including past rounds) allowing them to alter history and potentially create forks in the subsequent blockchain. Future quantum threats to the integrity of VRFs are significant concern today. This underscores the necessity of designing VRF schemes that are compact, efficient and resilient against quantum adversaries.

Given the rising significance of quantum-resistant cryptography, research efforts have increasingly targeted the development of blockchain tools that remain secure against quantum attacks. LB-VRF, proposed in 2021 by Esgin et al. [22] is the first quantum-safe VRF scheme based on lattices, which is considered as a significant step toward practical post-quantum VRFs. However, it has a major limitation, as it requires continuous key updates. Each key pair can generate only one VRF output, which introduces additional complications to compensate for the restricted VRF functionality. In 2022, Buser et al. [13] designed X-VRF, a quantum-resistant VRF scheme built upon hash functions that circumvent the problem of LB-VRF at the expense of huge key generation time.

1.1 Our Contribution

Motivated by the work of Buser et al. [13], we concentrate on reducing the key generation time while allowing multiple VRF outputs from a single key pair. We propose a post-quantum VRF construction that achieves faster key generation while enabling the production of multiple VRF outputs from a single key pair.

We introduce XM-VRF, an efficient post-quantum key updatable VRF construction employing hash-based cryptography which is one of the latest families of post-quantum candidates renowned for its compactness. To be more precise, we integrate in our XM-VRF a multi-tree version of the post-quantum secure Extended Merkle Signature Scheme (XMSS) of [11] in our XM-VRF construction. In XMSS, each leaf of the main Merkle tree is generated using an L-tree. Each L-tree accumulates the l components of a WOTS⁺ [33] public key into a single value and accumulated at its root using a hash function. At each level of the Merkle tree and L-tree, random bitmasks are applied to the hash inputs, adding an extra layer of randomness and strengthening the security of the tree structure.

Efficient key generation. The primary benefit of our XM-VRF over existing solutions is its improved key generation time and its capability to produce multiple VRF outputs from single key pair. Table 2 presents a comparison of our XM-VRF with X-VRF [13] and other practical post-quantum VRFs. The main idea to reduce the key generation time in our construction is to reduce the height of the XMSS trees by splitting the height of the XMSS trees into different layers. The lowest XMSS tree signs the input as a message while each upper XMSS tree signs the root node of the XMSS tree directly below it. The root node of the topmost XMSS tree is embedded in the verification key of XM-VRF. The computation of root node of the top tree takes less time than the computation of the root node of the whole height XMSS tree. The verification key of XM-VRF contains the root node and key generation efficiency of XM-VRF is achieved by significantly reducing the time to compute the root node of the topmost XMSS tree.

Forward security. To ensure the security of prior rounds even if a user key is later compromised, blockchain protocols such as Algorand [26] rely on VRF schemes and Ouroboros Praos [18] rely on key evolving digital signatures. Generic constructions for achieving forward security can be seamlessly instantiated in the post-quantum setting by replacing these signatures. Our XM-VRF ensures forward security by employing a forward secure stateful pseudorandom generator (PRG) that updates its internal state after each use, preventing compromise of past outputs and thereby making it suitable for blockchain setting. A detailed description of the forward-secure stateful pseudorandom generator (PRG) is given in Appendix C.

Key-updatable VRF: Our XM-VRF autonomously updates its secret key during the algorithm Eval, performing this update in parallel with the VRF output generation to ensure uninterrupted state progression. This property enables continuous and efficient VRF evaluation without explicit key refresh operations, thereby reducing computational overhead and ensuring forward security across multiple evaluations. We use a stateful forward-secure PRG to generate the WOTS⁺ key pairs sequentially, thereby eliminating the need to store all key pairs in memory.

Uniqueness of XM-VRF: At first glance, the core structure may appear to be simply a multi-tree version of XMSS applied to a VRF, but we introduce several substantial changes to the underlying primitives in order to achieve stronger security guarantees. Bodaghi et al. [6] proposed an attack against the uniqueness property of WOTS⁺, which in turn leads to uniqueness violations in both XMSS and X-VRF. Hence, a straightforward integration of a multi-tree variant of XMSS into a VRF construction will suffer from uniqueness failure. We alter the function family used in WOTS⁺ so that WOTS⁺ regains the uniqueness property. In XM-VRF, we instantiate this modified WOTS⁺ within the XMSS framework, thereby ensuring the uniqueness of both XMSS and XM-VRF. Additionally, handling the state management for the various layer-specific XMSS trees is difficult, and we addressed this aspect in great detail.

Protocol for random committee selection in Algorand: Algorand [25] is a proof-of-stake blockchain achieving fast, secure, and decentralized consensus, where network participants collectively agree on the next valid block. Each user independently runs a VRF using their secret key and the current round information containing round number, phase number and random seed as input; if the VRF output is below a threshold determined by their stake, the user is selected as a committee member for that round. We present a protocol for random committee selection within the Algorand blockchain framework, leveraging XM-VRF to ensure unbiased, verifiable and stake-proportional participant selection.

Comparison of XM-VRF with other existing post quantum VRFs: We present the comparison in Table 2 which summarizes the efficiency and security features of the proposed XM-VRF alongside other post-quantum VRF constructions. We make this comparison based on the following key features:

OutputCount refers to the number of VRF outputs that can be generated using a single secret-verification key pair.

KeyLifeTime represents the duration for which all VRF outputs derived from a single secret-verification key pair remain valid. It is measured based on the block generation interval of the Algorand blockchain, indicating how long a key pair can be securely used before renewal. Since Algorand produces a new block approximately every 5 seconds, we assume that each VRF output is valid for a single block. Therefore, we set **KeyLifetime** to $\alpha = 5$ seconds for each VRF output.

KeyGenTime denotes the time required to generate the secret-verification key pair for the VRF.

ProofSize represents the combined size of all components that constitute the proof accompanying a VRF output, reflecting the communication overhead during verification.

SecurityBasis denotes underlying cryptographic hardness assumptions that form the foundation of the VRF's security.

KeyUpdatable indicates whether the VRF key pair is updated after each evaluation (positive) or remains constant across all output generations (negative).

We now define several key terms used in the comparison to provide a clearer

understanding of the parameters and metrics considered in evaluating different VRF constructions.

$\text{XMSSTree}(2^n)$ denotes the time to compute XMSS tree with 2^n leaf nodes

$\text{MerkleTree}(2^n)$ denotes the time to compute Merkle tree with 2^n leaf nodes

	XMSS Tree	Merkle Tree
ComputationCost	$2^n(l+1) - 1$ Hashes	$2^{n+1} - 1$ Hashes
AuthenticationPathSize	n strings from $\{0, 1\}^\lambda$	n strings from $\{0, 1\}^\lambda$

Table 1. Comparison of XMSS Tree and Merkle Tree of height n , l is the WOTS⁺ key size

p denotes the height of the hash chain in iVRF construction, $t_{\text{MatrixMult}}$ denotes the time to compute a matrix multiplication of dimension $(m+c+1) \times m$, t_{Hash} denotes the time to compute a hash function from $\{0, 1\}^*$ to $\{0, 1\}^{2\lambda}$, t_{LowMC} denotes the time to compute LowMC block cipher.

	X-VRF [13]	LB-VRF [22]	iVRF [21]	SL-VRF [13]	XM-VRF [Our Work]
OutputCount	2^n	1	2^n	practically unlimited	2^n
KeyLifeTime	$\alpha \cdot 2^n$ sec	α sec	$\alpha \cdot 2^n$ sec	practically unlimited	$\alpha \cdot 2^n$ sec
KeyGenTime	$\text{XMSSTree}(2^n)$	$t_{\text{MatrixMult}}$	$\text{MerkleTree}(2^n) + n \cdot p \cdot t_{\text{Hash}}$	t_{LowMC}	$d \cdot \text{XMSSTree}(2^{n/d})$
ProofSize	1 WOTS ⁺ signature + 1 XMSS Authentication Path + 1 index	1 element of ring $\mathcal{R}$ + 1 element of ring $\mathcal{R}^{m+c+1}$	1 binary string + 1 Merkle Authentication Path	1 KKW NIZK proof	d WOTS ⁺ signature + d XMSS Authentication Path
SecurityBasis	Collision resistant Hash	Module LWE and Module SIS	Collision resistant Hash	Onewayness of LowMC block cipher	Collision resistant Hash
KeyUpdatable	✗	✗	✗	✗	✓

Table 2. Comparison of the proposed XM-VRF with other existing post quantum VRFs in terms of efficiency, security features, and key generation time. n : Height of tree for tree based VRFs such as X-VRF, iVRF, XM-VRF, α : Life time of a VRF output in Algorand blockchain (Assuming that each VRF evaluation is valid for a single block), d : The number of layers in XM-VRF.

In SL-VRF [13] the evaluation relies on a PRF, ensuring pseudorandom outputs, while uniqueness and verifiability are achieved through the KKW NIZK [36] proof system. So SL-VRF can generate different VRF output from different input using the same key pair. Consequently the OutputCount of SL-VRF is practically unlimited so the KeyLifetime of SL-VRF is huge. The KeyGenTime of SL-VRF is quite low, since it only requires a single evaluation of the LowMC block cipher to generate the key pair. But its proof uses KKW NIZK, which has 20 times more than other VRF's proof size. LB-VRF [22] achieves efficient

KeyGenTime since it requires only a single matrix multiplication during key generation. However, its **KeyLifetime** and **OutputCount** are very limited, as it can produce only one VRF output per secret-verification key pair. The proof of LB-VRF contains an element of the polynomial ring $\mathcal{R} = \mathbb{Z}_q/(X^d + 1)$ and an element of the ring $\mathcal{R}^{m+c+1}$ where m is the rank of Module-SIS (Short Integer Solution), c is the rank of Module-LWE (Learning With Errors). So **ProofSize** of LB-VRF comparatively smaller. Now if we notice the hash based VRF constructions such as X-VRF [13], iVRF [21] and our work XM-VRF. The **OutputCount** remains identical across all constructions, and therefore, they share the same **KeyLifetime**. Key generation of iVRF need time to compute a whole Merkle tree of height n and also have to compute some hash functions from $\{0, 1\}^*$ to $\{0, 1\}^{2^\lambda}$. Key generation of X-VRF requires time to compute a whole XMSS tree of height n which is much bigger than that of iVRF as in XMSS we have to compute L-tree for each leaf nodes. But KeyGen of XM-VRF need time to compute only the d XMSS tree of height $\frac{n}{d}$ in parallel, which is the smallest from these two hash based VRF construction while also providing identical **KeyLifetime**. Observing the computation cost of the XMSS and Merkle trees from Table 1, XM-VRF have to compute only $d \cdot 2^{n/d}(l+1) - d$ hashes which is less than that of iVRF and X-VRF. For instance if we consider $n = 40, l = 10$ and $d = 4$ then KeyGen of XM-VRF have to compute $44 \cdot 2^{10} - 4$ hashes but KeyGen of X-VRF must compute $11 \cdot 2^{40} - 1$ hashes and KeyGen of iVRF must compute atleast $2^{41} - 1$ hashes. The proof of X-VRF contains one WOTS⁺ signature, one XMSS tree authentication path and an index, proof of iVRF contains one binary string and a Merkle authentication path and proof of XM-VRF contains d WOTS⁺ signature and d XMSS authentication path. In iVRF the Merkle tree of height n results in Merkle authentication path comprising n nodes. Similarly in X-VRF the XMSS tree of height n also yields XMSS authentication path containing n nodes. In XM-VRF we uses d XMSS tree of height $\frac{n}{d}$ which implies $d \cdot \frac{n}{d} = n$ nodes. Thus, we can observe that the proof sizes of these three VRF constructions are very close to each other and considerably smaller than the KKW NIZK [36] proof of SL-VRF. Hence, in terms of communication efficiency, the proposed XM-VRF achieves a standard and practical proof size, comparable to other lightweight post-quantum VRF constructions. If we examine the security of these VRF constructions, both X-VRF, iVRF and XM-VRF rely on the hardness of a collision-resistant hash function modeled in the random oracle setting. The security of LB-VRF is based on the hardness assumptions of the Module-LWE and Module-SIS problems, whereas SL-VRF derives its security from the one-wayness property of the LowMC [2] block cipher. Focusing on the **KeyUpdatable** property, the X-VRF, LB-VRF, iVRF, and SL-VRF constructions generate a secret-verification key pair during **KeyGen**, which remains unchanged across all VRF evaluations. So these VRF constructions are not key updatable so we denote these with ✕ symbol. But our XM-VRF is a key updatable VRF as it updates the secret key after each evaluation and we denote it with ✓ symbol. Recently, several isogeny-based post-quantum VRF constructions [37, 38] have been proposed; however, we do not include them in our comparison since their

underlying assumptions and efficiency characteristics differ significantly from our XM-VRF framework. There also exist several classical (pre-quantum) VRF constructions [39, 19, 34, 1] but we exclude them from our comparison as our focus is solely on post-quantum secure VRF schemes.

1.2 Technical Overview

We outline below our technical approach in constructing a post-quantum secure VRF that ensures pseudorandomness, uniqueness and efficient key generation with ability to produce multiple VRF outputs from a single key pair.

Merkle Tree: A Merkle tree [40] is a binary structure that enables efficient verification and accumulation of large datasets through hashing. Each leaf stores a data hash, internal nodes hash their children, and the root acts as a compact commitment. An authentication path includes sibling nodes from a leaf to the root for efficient verification.

XMSS: Our proposed XM-VRF construction is built upon the Extended Merkle Signature Scheme (XMSS) [11] which serves as its core cryptographic component. XMSS enables multiple VRF evaluations to be derived from a single secret-verification key pair of VRF. Specifically, XMSS tree contains multiple one-time signature WOTS⁺ [33] public keys, with each leaf node corresponding to one WOTS⁺ public key. The corresponding WOTS⁺ secret keys are used for signing, which in turn yield multiple VRF outputs. To compute the leaf nodes from the WOTS⁺ keys, we employ a tree structure L-tree. L-tree takes the l components of a WOTS⁺ public key as its leaf nodes and accumulate them into a single hash value. The root of the topmost XMSS tree is stored in XM-VRF verification key. The XMSS tree is generated through the algorithm **TreeHash** which incrementally builds the tree in a bottom-up manner. During this process, a stack is utilized to manage intermediate nodes, enabling efficient computation and storage of partial tree information without requiring the entire tree to be held in memory. The partial tree information is maintained within stacks and auxiliary data structures to facilitate efficient computation of authentication paths corresponding to specific indices. A stateful forward-secure PRG is employed to generate the WOTS⁺ key pairs sequentially, thereby eliminating the need to store all key pairs in memory. Specifically, a random seed is chosen and the PRG is iteratively applied to this seed to produce subsequent WOTS⁺ secret keys and the necessary random strings. A chaining function is then applied to these secret keys to obtain the corresponding WOTS⁺ public keys. Only the initial seed needs to be stored, significantly reducing memory requirements.

Multi layer XMSS: However, employing an XMSS tree of large height introduces a significant challenge, as computing the VRF verification key necessitates deriving the root node of the entire tree. For an XMSS tree of height h , this requires generating 2^h WOTS⁺ key pairs and constructing the complete tree, which consequently leads to substantial key generation time and rendering the approach impractical for blockchain applications. To address this, we employ XMSS trees

of smaller height arranged across multiple layers. More precisely, we split XMSS trees of height h into d layers of XMSS trees of height $\frac{h}{d}$. Therefore, instead of constructing a single XMSS tree of height h , we compute d smaller XMSS trees, each of height $\frac{h}{d}$ which can be generated in parallel. Since XMSS tree in each layer are independent to each other, now we compute only $2^{h/d}$ WOTS⁺ key pairs in d layers in parallel to complete the key generation process. These XMSS trees in each layer are connected through WOTS⁺ signatures such as WOTS⁺ signatures on the root nodes of lower-layer XMSS trees are computed using the WOTS⁺ secret key associated with the left most leaf node of the XMSS tree in the layer immediately above. And we store these WOTS⁺ signatures in each layer as $\sigma_1, \sigma_2, \dots, \sigma_{d-1}$ in secret key of XM-VRF. For computing authentication paths of the XMSS trees we store partial information of the current and next trees in states such as $\text{St}_{\text{cur},0}, \text{St}_{\text{cur},1}, \dots, \text{St}_{\text{cur},d-1}, \text{St}_{\text{next},0}, \text{St}_{\text{next},1}, \dots, \text{St}_{\text{next},d-1}$ inside secret key of XM-VRF. During the evaluation we update this states to obtain authentication paths efficiently. In the evaluation process, we also computing new WOTS⁺ key pairs in each layer to construct the next trees in each layer. And once XM-VRF have used all the $2^{h/d}$ WOTS⁺ keys in the current XMSS tree then we change XMSS trees in each layer except the top layer and update the required informations, signatures. During the key generation we compute a single XMSS tree in each layer and we denote it as current tree in each layer. During evaluation, we begin constructing the next set of XMSS trees in each layer. Once the current XMSS trees are fully utilized, all relevant information is transitioned to these new trees, while the corresponding procedures for key management and updates continue smoothly. We then set the next trees as current trees in each layer except the top layer XMSS tree and again we compute next trees for them and so on. Hence, our approach achieves efficient key generation in the XM-VRF scheme. Since each current tree will provide us $2^{h/d}$ VRF outputs so after repeating this current and next tree update process $2^{h-h/d}$ times we can get 2^h VRF outputs.

Key-updatable construction: Since our construction allows multiple VRF evaluations from a single secret-verification key pair, distinct WOTS⁺ secret keys are required for each evaluation. To derive a specific WOTS⁺ secret key from the initial seed, we employ a stateful forward-secure PRG. The resulting key information is then incorporated into the XM-VRF secret key and updated accordingly. The secret key of XM-VRF contains the random seeds for the current and next XMSS trees in each layer, as $S_{\text{cur},0}, S_{\text{cur},1}, \dots, S_{\text{cur},d-1}, S_{\text{next},0}, S_{\text{next},1}, \dots, S_{\text{next},d-1}$. Additionally, WOTS⁺ signatures are included in the secret key of the XM-VRF to establish connections between XMSS trees at successive layers. This information is also updated whenever the XMSS trees at each layer are refreshed after evaluation.

1.3 Related Works

Initially proposed by Micali, Rabin, and Vadhan, [41], VRFs have emerged as a fundamental cryptographic primitive with diverse applications. In [41], Micali et

al. build VRF upon the Goldreich–Goldwasser–Micali pseudorandom function construction to demonstrate a link between unique signatures and VRFs, combining the inherent unpredictability of unique signatures with their verifiability. The notion of VRF has been further explored by Lysyanskaya et al.[39] in 2002 and Dodis et al.[20] in 2005 each building on distinct number-theoretic assumptions. Lysyanskaya et al. [39] construct a Verifiable Random Function (VRF) from unique signature scheme by performing the transformation introduced by Micali et al. [41]. Additional VRF constructions based on number-theoretic assumptions are presented in [19, 32, 34, 1, 5]. In [10], Brakerski et al. introduced the concept of a weak VRF in which pseudorandomness is guaranteed only for randomly chosen inputs. Additional primitives related to VRF such as constrained VRF, simulatable VRF have been introduced in [15, 24].

In the context of quantum-safe constructions, the feasibility of lattice-based VRFs was demonstrated in [45, 29]. Goyal et al. [29] present a construction that relies on sophisticated tools such as constrained PRFs without providing a practical efficiency evaluation. In a subsequent work, Yang et al. [45] briefly mentioned their VRF construction derived from zero-knowledge proofs, but they did not present any formal security proof or performance evaluation for it. Nevertheless, the authors acknowledge that the proposed construction guarantees only trusted uniqueness, rendering it inadequate for blockchain-based applications. In addition, the approach appears impractical, as even the fundamental proofs in [45] incur communication overheads of several megabytes. Later Esgin et al. proposed a practical lattice-based construction LB-VRF in [22] which have some limitations as it generate only a only one VRF output per secret-verification key pair. In 2023, Esgin et al. [23] presented an improved lattice-based post-quantum VRF construction. Some hash-based constructions are proposed in [13, 21]. The construction by Buser et al. [13] relies on hash-based signature scheme XMSS and have huge key generation time. Esgin et al. in [21] gave an indexed VRF and a authenticated extended version of that. Recently, isogeny-based VRF constructions are proposed in [37, 38].

1.4 Organization

The rest of the paper is structured as follows. In Section 2, we introduce the necessary notations, review the fundamental definitions, theorems, present proposed security for the unforgeability of the underlying signature schemes such as WOTS⁺, XMSS and VRF. In Section 3, we present our XM-VRF construction with detailed algorithms, establish its correctness, storage analysis and prove its security in the random oracle model. In Section 4, we discuss applications of VRFs in blockchain and describe our protocol for random committee selection within the Algorand blockchain framework. Section 5 concludes the paper.

2 Preliminaries

Notation. Let $\lambda \in \mathbb{N}$ denotes the security parameter and $\log a$ stands for $\log_2 a$. For two positive integers a, b , let $r_{a,b} = (r_a, r_{a+1}, \dots, r_b)$ be a sub-string of

$r = r_{1,j} = (r_1, r_2, \dots, r_j)$ if $b > a$ and $r_{a,b}$ is an empty string if $b < a$. We write $x \xleftarrow{\$} X$ if x is randomly chosen from the set X using the uniform distribution. By $\mathcal{A}^{\mathcal{O}_P}$ we mean the adversary $\mathcal{A}$ having oracle access to an algorithm P . A function $\text{negl}(\cdot)$ is negligible if for every positive integer c , there exists an integer N such that $\text{negl}(\lambda) < 1/\lambda^c$ for all $\lambda > N$. $H : \{0, 1\}^{2\lambda} \rightarrow \{0, 1\}^\lambda$ is a hash function here. $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda \times \mathcal{K}_\lambda$ is a pseudorandom generator which extends a λ -bit string to $\lambda + t$ bit pseudorandom string where elements of $\mathcal{K}_\lambda$ are t bit string. $x||y$ denotes the concatenation of the strings x and y . Here **Stack** refers to a linear data structure governed by the Last-In, First-Out (LIFO) rule, which means that the most recently inserted element is the first to be retrieved. It supports two fundamental operations: **push**(N), which places an element N at the top of the **Stack**, and **pop**() removes the topmost element from the **Stack**. $N.\text{height}()$ denotes the height of the element N in the **Stack**.

2.1 One-time Signature WOTS⁺

We describe below a type one-time signature scheme $\text{WOTS}^+ = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Vrfy})$ of Andreas Hülsing [33] with message space $\mathcal{M} = \{0, 1\}^m$.

Setup(1^λ) $\rightarrow$ $\text{pp}_{\text{WOTS}^+}$: A trusted authority takes a security parameter λ as input here and performs the following steps:

- i. Choose message length m and parameter $w \in \mathbb{N}, w > 1$ and compute $l_1 = \left\lceil \frac{m}{\log w} \right\rceil$, $l_2 = \left\lceil \frac{\log(l_1(w-1))}{\log w} \right\rceil + 1$.
- ii. Sample a key space $\mathcal{K}_\lambda$ and pick a keyed function family $\mathcal{F}_\lambda = \{f_k : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda \mid k \in \mathcal{K}_\lambda\}$.
- iii. Set public parameter as $\text{pp}_{\text{WOTS}^+} = (m, w, l_1, l_2, \mathcal{K}_\lambda, \mathcal{F}_\lambda)$ and publishes $\text{pp}_{\text{WOTS}^+}$.

KeyGen($\text{pp}_{\text{WOTS}^+}$) $\rightarrow$ (sk, pk): A user runs this algorithm on input the public parameter $\text{pp}_{\text{WOTS}^+} = (m, w, l_1, l_2, \mathcal{K}_\lambda, \mathcal{F}_\lambda)$ and generates the signing-verification key pair (sk, pk) in the following way:

- i. Compute $l = l_1 + l_2$ and choose $l + w - 1$ random λ -bit strings. The first l many random λ -bit strings are used to construct $(\text{sk}_1, \text{sk}_2, \dots, \text{sk}_l)$ with $\text{sk}_i \in \{0, 1\}^\lambda$ for $i = 1, 2, \dots, l$ and the remaining $w - 1$ random λ -bit strings are used to construct $r = r_{1,w-1} = (r_1, r_2, \dots, r_{w-1})$ with $r_i \in \{0, 1\}^\lambda$ for $i = 1, 2, \dots, w - 1$.
- ii. Pick $k \xleftarrow{\$} \mathcal{K}_\lambda$, compute $\text{pk}_i = c_k^{w-1}(\text{sk}_i, r)$ for $i = 1, 2, \dots, l$ and set $\text{pk} = \{\text{aux}, (\text{pk}_1, \dots, \text{pk}_l)\}$ where $\text{aux} = (r, k) \in \{0, 1\}^{\lambda \times (w-1)} \times \mathcal{K}_\lambda$ and $c_k^i(x, r)$ is a chaining function defined iteratively in **Fig. 1**.
- iii. Set the secret key $\text{sk} = \{\text{aux}, (\text{sk}_1, \dots, \text{sk}_l)\}$.
- iv. Keep sk secret to itself and publishes pk .

Sign($\text{pp}_{\text{WOTS}^+}, \text{sk}, M$) $\rightarrow \sigma$: Taking input the public parameter $\text{pp}_{\text{WOTS}^+} = (m, w, l_1, l_2, \mathcal{K}_\lambda, \mathcal{F}_\lambda)$, a signing key $\text{sk} = \{\text{aux} = (r, k), (\text{sk}_1, \dots, \text{sk}_l)\}$, a message $M \in \{0, 1\}^m$, the signer produces a signature σ on M as follows:

$$\begin{aligned} c_k^i(x, s) &= x \text{ if } i = 0 \text{ and} \\ c_k^i(x, s) &= f_k(c_k^{i-1}(x, s) \oplus s_i) \text{ if } i > 0 \end{aligned}$$

Fig. 1. Chaining Function $c_k^i(x, s)$ which takes a string $x \in \{0, 1\}^\lambda$, an iteration counter i , key $k \in \mathcal{K}_\lambda$ and randomized element $s = s_{1,j} = (s_1, s_2, \dots, s_j) \in \{0, 1\}^{\lambda \times j}$ with $j \geq i$ as input and $f_k : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ is a keyed function from $\mathcal{F}_\lambda$.

- i. Compute base w representation of M as $M = M_1 M_2 \dots M_{l_1}$ where $M_i \in \{0, 1, \dots, w-1\}$ for $i = 1, 2, \dots, l_1$. This can be done as l_1 is set as $l_1 = \left\lceil \frac{m}{\log w} \right\rceil$. Calculate $C = \sum_{i=1}^{l_1} (w-1 - M_i)$.
- ii. Compute base w representation of C as $C_1 C_2 \dots C_{l_2}$ where $C_i \in \{0, 1, \dots, w-1\}$ for $i = 1, 2, \dots, l_2$. This can be done as l_2 is set as $l_2 = \left\lfloor \frac{\log(l_1(w-1))}{\log w} \right\rfloor + 1$.
- iii. Set $b = (b_1, b_2, \dots, b_l) = M_1 \dots M_{l_1} C_1 \dots C_{l_2}$. As l is set as $l = l_1 + l_2$.
- iv. Using pp and sk , generate the signature

$$\sigma = (\sigma_1, \dots, \sigma_l) = (c_k^{b_1}(\text{sk}_1, r), \dots, c_k^{b_l}(\text{sk}_l, r))$$

where $r = r_{1,w-1} = (r_1, r_2, \dots, r_{w-1})$ with $r_i \in \{0, 1\}^\lambda$ for $i = 1, 2, \dots, w-1$ and $k \in \mathcal{K}_\lambda$ are extracted from aux in sk .

- v. Send the message-signature pair (M, σ) over a public channel.

$\text{Vrfy}(\text{pp}_{\text{WOTS}^+}, \text{pk}, M, \sigma) \rightarrow 1/0$: The verifier checks the validity of the signature $\sigma = (\sigma_1, \dots, \sigma_l)$ on message $M \in \{0, 1\}^m$ using the verification key $\text{pk} = \{\text{aux} = (r, k), (\text{pk}_1, \dots, \text{pk}_l)\}$ as follows:

- i. Compute $b_1, b_2, \dots, b_l$ using base w representation $M_1 M_2 \dots M_{l_1}$ of M and base w representation $C_1 C_2 \dots C_{l_2}$ of $C = \sum_{i=1}^{l_1} (w-1 - M_i)$ exactly in the same manner as in algorithm Sign .
- ii. Check $\text{pk}_i \stackrel{?}{=} c_k^{w-1-b_i}(\sigma_i, r_{b_i+1, w-1})$ for $i = 1, 2, \dots, l$ where $r_{b_i+1, w-1} = (r_{b_i+1}, r_{b_i+2}, \dots, r_{w-1})$ with $r_{b_i+j} \in \{0, 1\}^\lambda$ for $j = 1, 2, \dots, w-1-b_i$ are extracted from aux in pk .
- iii. Output 1 if comparison succeeds, otherwise output 0.

Correctness: Note that, pk_i are computed by invoking chain function c_k iteratively $(w-1)$ times on $(\text{sk}_i, r = r_{1,w-1} = (r_1, r_2, \dots, r_{w-1}))$. Also σ_i are generated by running the chaining function c_k iteratively b_i times on $(\text{sk}_i, r = r_{1,w-1} = (r_1, r_2, \dots, r_{w-1}))$. During verification, pk_i are recomputed by applying chain function c_k iteratively $(w-1-b_i)$ times on $(\sigma_i, r_{b_i+1, w-1} = (r_{b_i+1}, r_{b_i+2}, \dots, r_{w-1}))$ where σ_i are generated during signature algorithm by running the chaining function c_k iteratively b_i times on $(\text{sk}_i, r = r_{1,w-1} = (r_1, r_2, \dots, r_{w-1}))$.

Suppose $\sigma = (\sigma_1, \dots, \sigma_l)$ is computed in the correct way as shown in algorithm $\text{WOTS}^+.\text{Sign}(\text{pp}_{\text{WOTS}^+}, \text{sk}, M)$ where $\text{pp}_{\text{WOTS}^+} \leftarrow \text{WOTS}^+.\text{Setup}(1^\lambda)$, $(\text{sk}, \text{pk}) \leftarrow \text{WOTS}^+.\text{KeyGen}(\text{pp}_{\text{WOTS}^+})$ and $M \in \mathcal{M}$. We explain the correctness of $\text{WOTS}^+ =$

(Setup, KeyGen, Sign, Vrfy) below by showing the following equation holds for each $i = 1, 2, \dots, l$.

$$c_k^{w-1-b_i}(\sigma_i, r_{b_i+1, w-1}) = \text{pk}_i$$

For simplicity, let us rename $r_{b_i+1, w-1} = (r_{b_i+1}, r_{b_i+2}, \dots, r_{w-1})$ as $(s_1, s_2, \dots, s_{w-1-b_i}) = s_{1, w-1-b_i} = s$ where $s_j = r_{b_i+j}$ for $j = 1, 2, \dots, w-1-b_i$. For $w-1-b_i > 0$, we have

$$\begin{aligned} c_k^{w-1-b_i}(\sigma_i, r_{b_i+1, w-1}) &= c_k^{w-1-b_i}(\sigma_i, s) \\ &= f_k(c_k^{w-2-b_i}(\sigma_i, s) \oplus s_{w-1-b_i}) \\ &= \underbrace{f_k(f_k(\dots f_k(c_k^0(\sigma_i, s) \oplus s_1) \dots \oplus s_{w-1-b_i}) \dots)}_{w-1-b_i \text{ times}} \\ &= f_k(f_k(\dots f_k(\sigma_i \oplus s_1) \dots \oplus s_{w-1-b_i}) \dots) \\ &= f_k(f_k(\dots f_k(\sigma_i \oplus r_{b_i+1}) \dots \oplus s_{w-1-b_i}) \dots) \\ &= f_k(f_k(\dots \underbrace{f_k(c_k^{b_i}(\text{sk}_i, r) \oplus r_{b_i+1}) \dots \oplus s_{w-1-b_i})}_{c_k^{b_i+1}(\text{sk}_i, r)} \dots) \\ &= \underbrace{f_k(f_k(\dots f_k(c_k^{b_i+1}(\text{sk}_i, r) \oplus s_2) \dots \oplus s_{w-1-b_i}) \dots)}_{w-2-b_i \text{ times}} \\ &= f_k(f_k(\dots \underbrace{f_k(c_k^{b_i+1}(\text{sk}_i, r) \oplus r_{b_i+2}) \dots \oplus s_{w-1-b_i})}_{c_k^{b_i+2}(\text{sk}_i, r)} \dots) \\ &\vdots \\ &= f_k(c_k^{w-2}(\text{sk}_i, r) \oplus s_{w-1-b_i}) \\ &= f_k(c_k^{w-2}(\text{sk}_i, r) \oplus r_{w-1}) \\ &= c_k^{w-1}(\text{sk}_i, r) = \text{pk}_i \end{aligned}$$

When $w-1-b_i = 0$, we get

$$c_k^{w-1-b_i}(\sigma_i, r_{b_i+1, w-1}) = c_k^0(\sigma_i, r_{b_i+1, w-1}) = \sigma_i = c_k^{b_i}(\text{sk}_i, r) = c_k^{w-1}(\text{sk}_i, r) = \text{pk}_i$$

establishing the correctness of the one time signature scheme WOTS^+ .

Theorem 1. *If $\mathcal{F}_\lambda = \{f_k : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda | k \in \mathcal{K}_\lambda\}$ is a collision resistant keyed function family, then $\text{WOTS}^+ = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Vrfy})$ is a unique signature scheme.*

Proof. Let us assume that $\text{WOTS}^+ = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Vrfy})$ is not a unique signature scheme. So $|\Sigma(m, \text{pk})| \neq 1$ where $\Sigma(m, \text{pk}) = \{\sigma : \text{WOTS}^+.\text{Vrfy}(\text{pp}, \text{pk}, m, \sigma) = 1\}$. Therefore assume that there exist two different WOTS^+ signatures σ, σ' of the message m such that

$$\text{WOTS}^+.\text{Vrfy}(\text{pp}, \text{pk}, m, \sigma) = 1 = \text{WOTS}^+.\text{Vrfy}(\text{pp}, \text{pk}, m, \sigma')$$

Since $(\sigma_1, \sigma_2, \dots, \sigma_l) = \sigma \neq \sigma' = (\sigma'_1, \sigma'_2, \dots, \sigma'_l)$, so there must exist some i for which $\sigma_i \neq \sigma'_i$.

Also we have $\text{WOTS}^+.\text{Vrfy}(\text{pp}, \text{pk}, m, \sigma) = 1 = \text{WOTS}^+.\text{Vrfy}(\text{pp}, \text{pk}, m, \sigma')$ which implies $c_k^{w-1-b_i}(\sigma_i, r_{b_i+1, w-1}) = \text{pk}_i = c_k^{w-1-b_i}(\sigma'_i, r_{b_i+1, w-1})$.

Since $\sigma_i \neq \sigma'_i$, so at some iteration of c_k the output will be equal for σ_i, σ'_i first time, say j . Therefore for $r_{b_i+1, w-1} = s = (s_1, s_2, \dots, s_{w-1-b_i})$

$$\begin{aligned} c_k^j(\sigma_i, s) &= c_k^j(\sigma'_i, s) \\ \implies f_k(c_k^{j-1}(\sigma_i, s) \oplus s_j) &= f_k(c_k^{j-1}(\sigma'_i, s) \oplus s_j) \end{aligned}$$

Since j is the first iteration of c_k where it outputs equally for σ_i and σ'_i , so

$$\begin{aligned} c_k^{j-1}(\sigma_i, s) &\neq c_k^{j-1}(\sigma'_i, s) \\ \implies c_k^{j-1}(\sigma_i, s) \oplus s_j &\neq c_k^{j-1}(\sigma'_i, s) \oplus s_j \end{aligned}$$

So we obtain collision for the function f_k , which contradicting the fact that $\mathcal{F}_\lambda = \{f_k : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda | k \in \mathcal{K}_\lambda\}$ is a collision resistant keyed function family.

So our assumption was wrong. Hence $\text{WOTS}^+ = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Vrfy})$ is a unique signature scheme. $\square$

Theorem 2. [33] *Let $\lambda, w, m \in \mathbb{N}$, $w, n = \text{poly}(\lambda)$, $\mathcal{F}_\lambda = \{f_k : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda | k \in \mathcal{K}_\lambda\}$ is a second preimage resistant (SPR), undetectable (UD), one-way (OW) keyed function family. Then the advantage of $\text{WOTS}^+ = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Vrfy})$ against an EU-CMA attack satisfies the following*

$$\text{Adv}_{\mathcal{A}, \text{WOTS}^+}^{\text{EU-CMA}}((\lambda, w, m); t, 1) \leq w \cdot \text{Adv}_{\mathcal{B}, \mathcal{F}_\lambda}^{\text{UD}}(t^*) + w \cdot l \cdot \max\{\text{Adv}_{\mathcal{C}, \mathcal{F}_\lambda}^{\text{OW}}(t'), w \cdot \text{Adv}_{\mathcal{D}, \mathcal{F}_\lambda}^{\text{SPR}}(t')\}$$

where $\mathcal{B}$ is an adversary against UD, $\mathcal{C}$ is an adversary against OW, $\mathcal{D}$ is an adversary against SPR with $l_1 = \left\lceil \frac{m}{\log w} \right\rceil$, $l_2 = \left\lceil \frac{\log(l_1(w-1))}{\log w} \right\rceil + 1$, $l_1 + l_2 = l$, $t' = t + 3lw$ and $t^* = t + 3lw + w - 1$.

As shown in Boneh et al. [7], WOTS^+ satisfies SU-CMA, stronger notion of EU-CMA, where the adversary can forge a message m^* from query list Qlist but with a different valid signature i.e., the queried signature σ on m^* is different from forged signature σ^* on m^* submitted by the adversary where $m^* \in \text{Qlist}$. Further details on these security notions, such as EU-CMA, are provided in Appendix A, while descriptions of different function families, including OW, SPR and UD, are given in Appendix B.

2.2 Extended Merkle Signature Scheme XMSS

We recall the extended Merkle signature scheme $\text{XMSS} = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Vrfy})$ of Buchmann et al. [11]. It uses one time signature WOTS^+ as the underlying collision resilient signature scheme and invokes a procedure XMSS Tree that assigns WOTS^+ public keys at the leaves.

XMSS Tree: An XMSS tree of height h is a binary tree having $h+1$ levels with the leaves at level 0 and the root at level h . The nodes at level j are denoted by $N_{i,j}$, $0 \leq i < 2^{h-j}$, $0 \leq j \leq h$. Taking 2^h many WOTS⁺ public keys as input, the XMSS Tree authenticates them by assigning at leaf nodes $N_{i,0}$, $1 \leq i \leq 2^h$ and accumulate them at the root node $rt = N_{0,h}$ as shown in **Algorithm 1** using sub-procedure L-tree described in **Algorithm 2**. A pictorial representation of XMSS tree with $h = 3$ is shown in **Fig. 2**.

Algorithm 1: XMSS Tree

Public Parameter: $pp = pp_{XMSS} = (pp_{WOTS^+}, k, h, G, H)$ where
 $pp_{WOTS^+} = (m, w, l_1, l_2, \mathcal{K}_\lambda, \mathcal{F}_\lambda) \leftarrow WOTS^+.Setup(1^\lambda)$ and $k \in \{0, 1\}^\lambda$, $h \in \mathbb{N}$,
 $G: \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$ is a PRG, $H: \{0, 1\}^{2\lambda} \rightarrow \{0, 1\}^\lambda$ is a hash function.
Input: $pk^{(1)}, pk^{(2)}, pk^{(3)}, \dots, pk^{(2^h)}, B_1, B_2, \dots, B_{h+\lceil \log l \rceil}$ where $pk^{(i)} = \{aux^{(i)}, (pk_1^{(i)}, pk_2^{(i)}, \dots, pk_l^{(i)})\}$ for $i = 1, 2, \dots, 2^h$ and $B_i \in \{0, 1\}^{2\lambda}$ for $i = 1, 2, \dots, h + \lceil \log l \rceil$
Output: rt
1. **for** ($i = 1; i \leq 2^h; i++$) **do**
2. $rt^{(i)} = \text{L-tree}(pp_{WOTS^+}, H, pk_1^{(i)}, pk_2^{(i)}, \dots, pk_l^{(i)}, B_{h+1}, B_{h+2}, \dots, B_{h+\lceil \log l \rceil})$;
3. Set $N_{i-1,0} = rt^{(i)}$;
4. **endfor**
5. **for** ($j = 1; j \leq h; j++$) **do**
6. **for** ($i = 0; i < 2^{h-j}; i++$) **do**
7. $N_{i,j} = H((N_{2i,j-1} || N_{2i+1,j-1}) \oplus B_j)$;
8. **endfor**
9. **endfor**
10. **Return** $rt = N_{0,h}$;

L-tree: An L-tree is a binary hash tree which takes l many λ -bit strings $(pk_1, pk_2, \dots, pk_l)$ from public key $\{aux, (pk_1, \dots, pk_l)\}$ of WOTS⁺ as input and condense it to a single λ -bit string as in **Algorithm 2**. **Fig. 3** provides a comprehensive visual representation of L-tree having $l = 5$ leaf nodes.

The Extended Merkle Signature Scheme XMSS comprises of four probabilistic polynomial time (PPT) algorithms (Setup, KeyGen, Sign, Vrfy) having the following interface:

Setup(1^λ) $\rightarrow pp_{XMSS}$: A trusted authority takes as input a security parameter λ and produces public parameter pp_{XMSS} in subsequent manner:

- i. Run $WOTS^+.Setup(1^\lambda) \rightarrow pp_{WOTS^+} = (m, w, l_1, l_2, \mathcal{K}_\lambda, \mathcal{F}_\lambda)$ where m is the length of the message, w is the parameter, $\mathcal{K}_\lambda$ is key space, $\mathcal{F}_\lambda = \{f_k : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda \mid k \in \mathcal{K}_\lambda\}$ is a keyed pseudorandom function family, $l_1 = \left\lceil \frac{m}{\log w} \right\rceil$ and $l_2 = \left\lceil \frac{\log(l_1(w-1))}{\log w} \right\rceil + 1$.

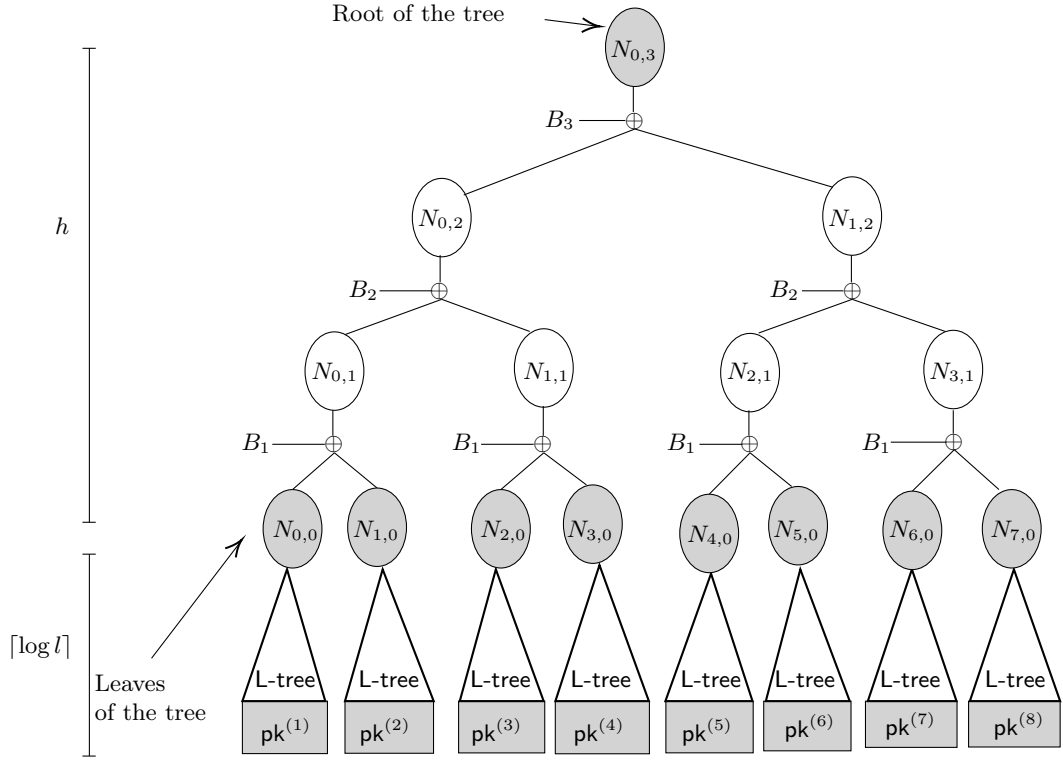


Fig. 2. Example of XMSS tree with $h = 3$ where $N_{i,j} = H((N_{2i,j-1} || N_{2i+1,j-1}) \oplus B_j)$ and $rt^{(i)} = \text{L-tree}(\text{pk}^{(i)}, B_{h+1}, B_{h+2}, \dots, B_{h+\lceil \log l \rceil})$

- ii. Pick $h \in \mathbb{N}$ and a pseudorandom generator $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda \times \mathcal{K}_\lambda$ and a second preimage resistant hash function $H : \{0, 1\}^{2\lambda} \rightarrow \{0, 1\}^\lambda$.
- iii. Select $k \in \mathcal{K}_\lambda$ for the key in function family.
- iv. Publish the public parameter $\text{pp}_{\text{XMSS}} = (\text{pp}_{\text{WOTS}^+}, k, h, G, H)$.

KeyGen(pp_{XMSS}) $\rightarrow$ (sk, pk): A user takes public parameter $\text{pp}_{\text{XMSS}} = (\text{pp}_{\text{WOTS}^+}, k, h, G, H)$ as input and generates XMSS signing-verification key pair (sk, pk) for XMSS by executing the following steps:

- i. Parse $\text{pp}_{\text{WOTS}^+} = (m, w, l_1, l_2, \mathcal{K}_\lambda, \mathcal{F}_\lambda)$
- ii. Choose $s_0 \in \{0, 1\}^\lambda$ and set $l = l_1 + l_2$.
- iii. **for** ($i = 1; i \leq 2^h; i++$) **do**
 Run $G(s_{i-1}) = s_i || t_i$ where $s_i \in \{0, 1\}^\lambda$ and $t_i \in \mathcal{K}_\lambda$
 Set λ -bit string $\text{rand}_j = f_{t_i}(\text{bin}(j-1))$ for $j = 1, \dots, l+w-1$ where $\text{bin}(j-1)$ is binary representation of $j-1$ and $f_{t_i} \in \mathcal{F}_\lambda$
 Use $\lambda(l+w-1)$ -bit string $\text{rand} = (\text{rand}_1, \dots, \text{rand}_{l+w-1})$ to construct random element $r^{(i)} = (\text{rand}_{l+1}, \dots, \text{rand}_{l+w-1})$ and

Algorithm 2: L-tree

Input: $\text{pp} = \text{pp}_{\text{WOTS}^+} = (m, w, l_1, l_2, \mathcal{K}_\lambda, \mathcal{F}_\lambda), \text{H}, (\text{pk}_1, \text{pk}_2, \dots, \text{pk}_l)$ and $(B_1, B_2, \dots, B_{\lceil \log l \rceil})$ where $\text{pk}_i \in \{0, 1\}^\lambda$ for $i = 1, \dots, l, B_i \in \{0, 1\}^{2^\lambda}$ for $i = 1, \dots, \lceil \log l \rceil, \text{H} : \{0, 1\}^{2^\lambda} \rightarrow \{0, 1\}^\lambda$ is a hash function and $\text{pp}_{\text{WOTS}^+} \leftarrow \text{WOTS}^+. \text{Setup}(1^\lambda)$

Output: rt

1. **for** $(i = 0; i < l; i++)$ **do**
2. $L_{i,0} = \text{pk}_{i+1};$
3. **endfor**
4. **for** $(j = 1; j \leq \lceil \log l \rceil; j++)$ **do**
5. **for** $(i = 0; i < \lfloor \frac{l}{2^j} \rfloor; i++)$ **do**
6. $L_{i,j} = \text{H}((L_{2i,j-1} || L_{2i+1,j-1}) \oplus B_j);$
7. **endfor**
8. **endfor**
9. **if** $(l \text{ is not a power of } 2)$ **then**
10. A node with no right sibling is pushed to a higher level of the tree until it
11. becomes the right sibling of another node i.e.,
12. **for** $(j = 1; j \leq \lceil \log l \rceil - 1; j++)$ **do**
13. **if** $(\lceil \frac{l}{2^{j+1}} \rceil \equiv 1 \pmod{2})$ **then**
14. $L_{\lfloor \frac{l}{2^j} \rfloor, j} = L_{\lfloor \frac{l}{2^{j+1}} \rfloor, j-1};$
15. **endif**
16. **endfor**
17. **endif**
18. **Return** $\text{rt} = L_{0, \lceil \log l \rceil};$

$\text{sk}^{(i)} = \{(r^{(i)}, k), (\text{sk}_1, \dots, \text{sk}_l)\}$ where $\text{sk}_j = \text{rand}_j$ for $j = 1, \dots, l$
Set $\text{pk}_j = c_k^{w-1}(\text{sk}_i, r^{(i)})$ for $j = 1, \dots, l$ where $c_k^i(x, r)$ is a chaining function defined iteratively in **Fig. 1**.
Assign $\text{pk}^{(i)} = \{(r^{(i)}, k), (\text{pk}_1, \dots, \text{pk}_l)\}$

endfor

- iv. Pick random bitmasks $B_j \in \{0, 1\}^{2^\lambda}$ for $j = 1, \dots, h + \lceil \log l \rceil$
- v. Run $\text{XMSS Tree}(\text{pp}_{\text{XMSS}}, \text{pk}^{(1)}, \dots, \text{pk}^{(2^h)}, B_1, B_2, \dots, B_{h+\lceil \log l \rceil}) \rightarrow N_{0,h}$
- vi. Set the secret key $\text{sk} = s_0$ and the public key $\text{pk} = (N_{0,h}, B_1, B_2, \dots, B_{h+\lceil \log l \rceil}, \{r^{(i)}\}_{i=1}^{2^h})$
- vii. Keep sk secret to itself and publish pk .

$\text{Sign}(\text{pp}_{\text{XMSS}}, \text{sk}, i, M) \rightarrow \text{Sig}$: A signer on input the public parameter pp_{XMSS} , signing key sk , index i and the message $M \in \{0, 1\}^m$ to generate the signature Sig on M as follows:

- i. Parse $\text{sk} = s$
- for** $(j = 1; j \leq i; j++)$ **do**
- $G(s) = s || t$ where $s \in \{0, 1\}^\lambda$ and $t \in \mathcal{K}_\lambda$
- endfor**
- ii. Set λ -bit string $\text{rand}_j = f_t(\text{bin}(j-1))$ for $j = 1, \dots, l + w - 1$ where $\text{bin}(j-1)$ is binary representation of $j-1$ and $f_t \in \mathcal{F}_\lambda$

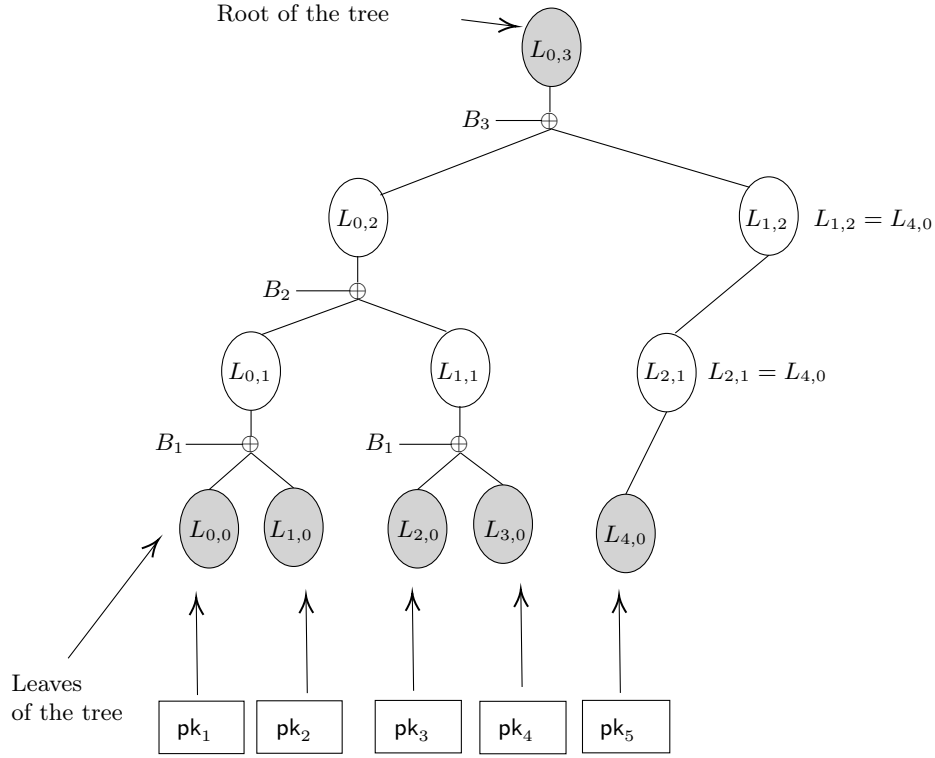


Fig. 3. Example of L-tree with $l = 5$ leaf nodes where $L_{i,j} = H((L_{2i,j-1} || L_{2i+1,j-1}) \oplus B_j)$

- Use $\text{rand} = (\text{rand}_1, \dots, \text{rand}_{l+w-1}) \in \{0, 1\}^{\lambda \times (l+w-1)}$ to construct random element $r^{(i)} = (\text{rand}_{l+1}, \dots, \text{rand}_{l+w-1}) \in \{0, 1\}^{\lambda \times (w-1)}$ and set $\text{sk}_j = \text{rand}_j$ for $j = 1, \dots, l$
- Set $\text{pk}_j = c_k^{w-1}(\text{sk}_j, r^{(i)})$ for $j = 1, \dots, l$ where $c_k^i(x, r)$ is a chaining function defined iteratively in **Fig. 1**
- Assign $\text{sk}^{(i)} = \{(r^{(i)}, k), (\text{sk}_1, \dots, \text{sk}_l)\}$ and $\text{pk}^{(i)} = \{(r^{(i)}, k), (\text{pk}_1, \dots, \text{pk}_l)\}$
- iii. Generate signature $\sigma \leftarrow \text{WOTS}^+. \text{Sign}(\text{pp}_{\text{WOTS}^+}, \text{sk}^{(i)}, M)$ where $\sigma = (\sigma_1, \dots, \sigma_l) = (c_k^{b_1}(\text{sk}_1, r^{(i)}), \dots, c_k^{b_l}(\text{sk}_l, r^{(i)}))$
 - iv. Calculate the authentication path $\text{Auth} = (\text{Auth}_0, \dots, \text{Auth}_{h-1})$ (contains the sibling nodes corresponding to each node along the path from the leaf node $N_{0,i}$ to the root) of the i -th leaf of the XMSS tree using AuthCalc algorithm written in **Algorithm 4**.
 - v. Send the signature $\text{Sig} = (i, \sigma, \text{Auth})$

$\text{Vrfy}(\text{pp}_{\text{XMSS}}, \text{pk}, M, \text{Sig}) \rightarrow 1/0$: The verifier checks the validity of the signature Sig on message M using the verification key pk in the following method:

- i. Parse $\text{Sig} = (i, \sigma, \text{Auth})$.

- ii. Compute $b_1, b_2, \dots, b_l$ using base w representation $M_1 \dots M_{l_1}$ of M and calculating base w representation $C_1 C_2 \dots C_{l_2}$ of $C = \sum_{j=1}^{l_1} (w - 1 - M_j)$ exactly in the same manner as in algorithm $\text{WOTS}^+.\text{Sign}$.
- iii. Extract $r^{(i)}, k$ from pk , σ from Sig
Set $\text{pk}_j^{(i)} = c_k^{w-1-b_j}(\sigma_j, r_{b_j+1, w-1}^{(i)})$ for $j = 1, \dots, l$
- iv. Run $\text{L-tree}(\text{pp}_{\text{WOTS}^+}, \text{pk}_1^{(i)}, \text{pk}_2^{(i)}, \dots, \text{pk}_l^{(i)}, B_{h+1}, B_{h+2}, \dots, B_{h+\lceil \log l \rceil}) \rightarrow \text{rt}^{(i)}$. Set the corresponding XMSS tree leaf node as $\text{rt}^{(i)}$. Then use this leaf $\text{rt}^{(i)}$ and the authentication path $\text{Auth} = (\text{Auth}_0, \dots, \text{Auth}_{h-1})$ to compute the root node rt of the XMSS tree.
- v. Extract $N_{0,h}$ from pk .
Check $\text{rt} \stackrel{?}{=} N_{0,h}$
Output 1, if verification passes, otherwise output 0

Theorem 3. [11] If $H : \{0, 1\}^{2\lambda} \rightarrow \{0, 1\}^\lambda$ is a second preimage resistant hash function and $\mathcal{F}_\lambda = \{f_k : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda \mid k \in \mathcal{K}_\lambda\}$ is a pseudorandom function family, then $\text{XMSS} = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Vrfy})$ is EU-CMA secure.

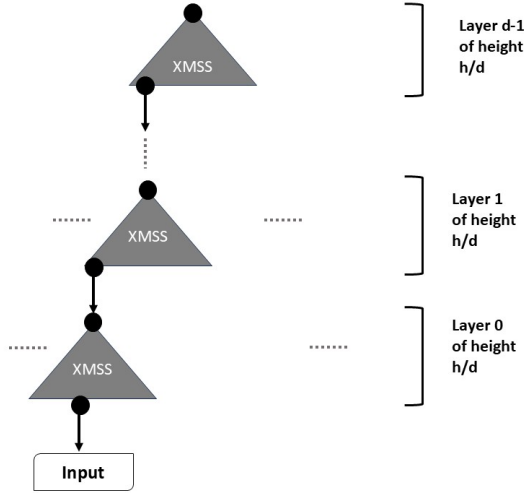


Fig. 4. XMSS tree with d layers

2.3 Verifiable Random Function(VRF)

A verifiable random function (VRF) is a function that generates a pseudo-random value that preserves the verifiability property where anyone can verify the evaluated pseudo-random value using a proof provided by the VRF.

Definition 1 (Verifiable Random Function). [41] A VRF = (ParamGen, KeyGen, Eval, Verify) with input length l and output length m consists of the following polynomial time algorithms:

ParamGen(1^λ) $\rightarrow$ pp_{VRF} : On input a security parameter λ , this probabilistic algorithm run by a trusted authority, outputs global public parameter pp_{VRF} .

KeyGen(pp_{VRF}) $\rightarrow$ ($\text{sk}_{\text{VRF}}, \text{vk}_{\text{VRF}}$): On input public parameter pp_{VRF} this probabilistic algorithm run by the evaluator, generates a secret key sk_{VRF} and a verification key vk_{VRF} .

Eval($\text{pp}_{\text{VRF}}, \text{sk}_{\text{VRF}}, x$) $\rightarrow$ ($y_{\text{VRF}}, \pi_{\text{VRF}}$): The evaluator with its secret key sk_{VRF} and an input x of length l this algorithm outputs the evaluation value y_{VRF} and the corresponding proof π_{VRF} of correct computation of y_{VRF} .

Verify($\text{pp}_{\text{VRF}}, \text{vk}_{\text{VRF}}, x, y_{\text{VRF}}, \pi_{\text{VRF}}$) $\rightarrow$ $1/0$: On input ($\text{pp}_{\text{VRF}}, \text{vk}_{\text{VRF}}, y_{\text{VRF}}, x, \pi_{\text{VRF}}$) a verifier executes this probabilistic algorithm and outputs either 1 or 0.

A VRF is required to have the following security properties:

Provability: If ($y_{\text{VRF}}, \pi_{\text{VRF}}$) is the output of $\text{VRF.Eval}(\text{pp}_{\text{VRF}}, \text{sk}_{\text{VRF}}, x)$ where $\text{sk}_{\text{VRF}}, \text{vk}_{\text{VRF}}$ are honestly generated by the algorithm VRF.KeyGen , then VRF = (ParamGen, KeyGen, Eval, Verify) is said to satisfy provability if

$$\text{VRF.Verify}(\text{pp}_{\text{VRF}}, \text{vk}_{\text{VRF}}, x, y_{\text{VRF}}, \pi_{\text{VRF}}) = 1$$

Uniqueness: VRF = (ParamGen, KeyGen, Eval, Verify) is said to satisfy uniqueness if no values ($\text{vk}_{\text{VRF}}, y_{\text{VRF}}^{(1)}, y_{\text{VRF}}^{(2)}, x, \pi_{\text{VRF}}^{(1)}, \pi_{\text{VRF}}^{(2)}$) can satisfy

$$\text{VRF.Verify}(\text{pp}_{\text{VRF}}, \text{vk}_{\text{VRF}}, x, y_{\text{VRF}}^{(1)}, \pi_{\text{VRF}}^{(1)}) = \text{VRF.Verify}(\text{pp}_{\text{VRF}}, \text{vk}_{\text{VRF}}, x, y_{\text{VRF}}^{(2)}, \pi_{\text{VRF}}^{(2)}) = 1$$

when $y_{\text{VRF}}^{(1)} \neq y_{\text{VRF}}^{(2)}$

Pseudorandomness: Let $\mathcal{A}$ be a probabilistic polynomial time (PPT) adversary participating in the experiment $\text{Exp}_{\mathcal{A}, \text{VRF}}^{\text{PRand}}(\lambda)$ in **Experiment 4**.

VRF = (ParamGen, KeyGen, Eval, Verify) is said to satisfy pseudorandomness if

$$\text{Adv}_{\mathcal{A}, \text{VRF}}^{\text{PRand}}(\lambda) = \max_{\mathcal{A}} \left\{ \Pr \left[\text{Exp}_{\mathcal{A}, \text{VRF}}^{\text{PRand}}(\lambda) = 1 \right] \right\} \leq \frac{1}{2} + \text{negl}(\lambda)$$

Experiment 4: $\text{Exp}_{\mathcal{A}, \text{VRF}}^{\text{PRand}}(\lambda)$

Setup: The challenger generates public parameter $\text{pp}_{\text{VRF}} \leftarrow \text{VRF.ParamGen}(1^\lambda)$ by taking λ as input. Then challenger produces key pairs ($\text{sk}_{\text{VRF}}, \text{vk}_{\text{VRF}}$) $\leftarrow \text{VRF.KeyGen}(\text{pp}_{\text{VRF}})$. It maintains a list Qlist and initializes Qlist = ϕ . It gives the public parameter pp_{VRF} and the verification key vk_{VRF} to adversary $\mathcal{A}$ and keep sk_{VRF} secret to itself.

Query Phase: The adversary $\mathcal{A}$ repeats polynomial times evaluation query on inputs of its choice to obtain VRF outputs from the challenger. For a evaluation query on an input $x \in \{0, 1\}^l$ from $\mathcal{A}$, the challenger generates $\text{VRF.Eval}(\text{pp}_{\text{VRF}}, \text{sk}_{\text{VRF}}, x) \rightarrow (y, \pi)$ and send it to $\mathcal{A}$ and update Qlist = Qlist $\cup \{x\}$.

Challenge: After all these queries $\mathcal{A}$ submit an input x^* and a secret state st

to the challenger using $\mathcal{A}^{\mathcal{O}_{\text{VRF.Eval}(\cdot)}}(\text{vk}_{\text{VRF}}) \rightarrow (x^*, st)$. The challenger in turn generates $(y_0, \pi_0) \leftarrow \text{VRF.Eval}(\text{pp}_{\text{VRF}}, \text{sk}_{\text{VRF}}, x^*)$ and challenge $y_1 \leftarrow \{0, 1\}^m$. Then the challenger send y_b to the adversary $\mathcal{A}$ where $b \xleftarrow{\$} \{0, 1\}$.

Response: In response adversary $\mathcal{A}$ sends b' to the challenger. If $b = b'$ and $x^* \notin \text{Qlist}$, then return 1, otherwise return 0.

where $\mathcal{O}_{\text{VRF.Eval}(\cdot)}$ is an oracle that on an input value x outputs the VRF value y_{VRF} and the corresponding proof π_{VRF} of the correct computation of y_{VRF} .

Algorithm 3: TreeHash

Input: stack **Stack**, node N , hash function $H : \{0, 1\}^{2\lambda} \rightarrow \{0, 1\}^\lambda$, random bitmasks $B_i \in \{0, 1\}^{2\lambda}$ for $i = 1, \dots, h + \lceil \log l \rceil$
Output: updated stack **Stack**
1. **while** (top node on **Stack** has same height as N) **do**
2. $t \leftarrow N.\text{height}() + 1$;
3. $N \leftarrow H((\text{Stack.pop}() || N) \oplus B_t)$;
4. **endwhile**
5. **Stack.push**(N);
6. **return** **Stack**;

3 Construction of XM-VRF from XMSS

This section introduces our construction of a hash based post-quantum secure VRF named XM-VRF = (ParamGen, KeyGen, Eval, Verify) uses XMSS (described in section 2.2) of height $h' = h/d$ in d layers as shown in **Fig. 4**. Informally, our XM-VRF employs a hierarchical XMSS structure, the bottom XMSS tree signs the actual input as message, and each higher-level XMSS tree signs the root of the tree immediately beneath it. The verification key of XM-VRF contains the root node of the top layer XMSS tree. Sub-procedures namely TreeHash, AuthCalc, WOTS⁺PKfromSign and Auth2Root presented in **Algorithm 3**, **Algorithm 4**, **Algorithm 6** and **Algorithm 5** respectively are employed by our XM-VRF. Our XM-VRF construction allows multiple iterations of VRF evaluations within a single round (i.e, one step of consensus), matching the Algorand [25] setting. The algorithms for the VRF given in sub section 3.1 below. Prior to describing the main algorithms, we define four auxiliary sub-procedures that will be employed throughout the construction.

- The sub-procedure TreeHash is described in **Algorithm 3**. It primarily constructs XMSS trees using a stack-based approach along with node information. When we call this sub-procedure, it takes a node and a **Stack** (containing essential tree nodes information in some states) as input and update the informations stored in **Stack**. **Stack** refers to a linear data structure governed by the Last-In, First-Out (LIFO) rule — meaning the most recently inserted element is the first to be retrieved. It supports two fundamental operations: **push**(N), which places an element N at the top of the **Stack**, and **pop**() removes the topmost element from the **Stack**. $N.\text{height}()$ denotes the height

Algorithm 4: AuthCalc

Input: $\text{index} \in \{1, 2, \dots, 2^{h'}\}, h, \text{ac} \geq 2, \text{St}_{\text{cur}, i}, H : \{0, 1\}^{2^\lambda} \rightarrow \{0, 1\}^\lambda$
Output: The updated state $\text{St}_{\text{cur}, i}$

1. Extract $\{\text{Auth}_i\}_{i=0}^{h-1}$ from $\text{St}_{\text{cur}, i}$
2. $\text{lp} \leftarrow \max\{g | 2^g | (\text{index} + 1)\}$;
3. **if** $(\text{lp} \stackrel{g \geq 0}{\leq} h - 1 \text{ and } \lfloor \text{index} / 2^{\text{lp}+1} \rfloor \text{ is even })$ **then**
4. $\text{Store}_{\text{lp}} \leftarrow \text{Auth}_{\text{lp}}$;
5. **endif**
6. **if** $(\text{lp} = 0)$ **then**
7. $\text{Auth}_0 \leftarrow \text{LeafCalc}(\text{index})$;
8. **endif**
9. **if** $(\text{lp} > 0)$ **then**
10. $\text{Auth}_{\text{lp}} \leftarrow H(\text{Auth}_{\text{lp}-1} || \text{Store}_{\text{lp}-1})$;
11. Remove $\text{Store}_{\text{lp}-1}$;
12. **for** $(i = 0; i \leq \text{lp} - 1; i++)$ **do**
13. **if** $(i \leq h - \text{ac} - 1)$ **then**
14. $\text{Auth}_i \leftarrow \text{Stock}_i$;
15. **else**
16. $\text{Auth}_i \leftarrow \text{Cache}_i.\text{pop}()$;
17. **endif**
18. **endfor**
19. **for** $(i = 0; i \leq \min\{\text{lp} - 1, h - \text{ac} - 1\}; i++)$ **do**
20. **if** $(\text{index} + 1 + 3 \cdot 2^i < 2^h)$ **then**
21. $\text{TreeHash}_i.\text{init}(\text{index} + 1 + 3 \cdot 2^i)$;
22. **endif**
23. **endfor**
24. **endif**
25. **for** $(i = 1; i \leq \frac{h-\text{ac}}{2}; i++)$ **do**
26. $s \leftarrow \min\{j | \text{Treehash}_j.\text{height} = \min_{k=0, \dots, h-\text{ac}-1} \text{Treehash}_k.\text{height}\}$;
27. $\text{Treehash}_s.\text{update}()$;
28. **endfor**

of the element N in the **Stack**. Finally **Stack** contains only the root node of the XMSS tree after calling **TreeHash** sub-procedure iteratively. XM-VRF employs this sub-procedure in algorithms **VRF.KeyGen** and **VRF.Eval**.

- The sub-procedure **AuthCalc** is described in **Algorithm 4**. This sub-procedure efficiently computes authentication paths of the XMSS tree in an optimal manner. **AuthCalc** takes leaf index index , height h , parameter ac , hash function H and state St as input and generates authentication path for next leaf index and update that information in the state St . The core idea is that the left child of a node can be derived from values that already appear in the authentication path. It leverages the stored node information from the states maintained and updated by the **TreeHash** sub-procedure. These state contains **Store**, **Stock**, **Cache**, **Auth**. We describe these data structures in the following:

Algorithm 5: Auth2Root

Input: Public parameter $\text{pp}_{\text{XM-VRF}} = (\text{ctr}, m, d, h, \text{ac}, w, \mathcal{F}_\lambda, H, H_1, G)$, authentication path Auth , leaf node leaf , bitmasks $B_1, \dots, B_{h'}$
Output: Root node rt

1. Compute $h' = \frac{h}{d}$;
2. Parse $\text{Auth} = (P_1, \dots, P_{h'})$ where $P_i \in \{0, 1\}^\lambda$ for $i = 1, 2, \dots, h'$;
 Set $L = \text{leaf} \in \{0, 1\}^\lambda$;
for $(i = 1; i \leq h'; i++)$ **do**
 Compute $L = H((P_i || L) \oplus B_i) \in \{0, 1\}^\lambda$;
endfor
3. **return** L ;

- **Store** maintains selected nodes that are used to efficiently compute left authentication nodes.
- **Stock** stores a single tail node, which is the first node that sub-procedure **TreeHash** pushes onto the stack **Stack**. The remaining tail nodes are subsequently pushed onto the stack **Stack**.
- **Cache** maintains a stack of right authentication nodes that assist in the computation of the desired authentication path.
- **Auth** is a collection of nodes that store the current authentication path.
- Here $\text{TreeHash}_i.\text{init}()$ initializes the instance with the leaf index index from which to begin.
- $\text{Treehash}_j.\text{height}$ stores the height of the lowest tail node maintained by the sub-procedure **TreeHash** instance, either on the stack **Stack** or in the entry **Stock**. If sub-procedure **TreeHash** does not maintain any tail nodes, then $\text{Treehash}_j.\text{height}$ is set to j . If sub-procedure **TreeHash** is either finished or uninitialized, then $\text{Treehash}_j.\text{height}$ is set to ∞ .
- $\text{Treehash}_s.\text{update}()$ executes sub-procedure **TreeHash** once, which means it first computes the next leaf, and then performs the necessary hash function evaluations to compute the leaf's parent nodes, provided that tail nodes are stored on the stack **Stack**.

XM-VRF employs this sub-procedure in algorithm **VRF.Eval**.

- The sub-procedure $\text{WOTS}^+ \text{PKfromSign}$ is described in **Algorithm 6**. The purpose of this algorithm is to reconstruct the WOTS^+ public key pk from the corresponding WOTS^+ signature σ . XM-VRF employs this sub-procedure in algorithm **VRF.Verify**.
- Finally, the sub-procedure **Auth2Root** is described in **Algorithm 5**. This sub-procedure computes the root node rt of an XMSS tree using a given leaf node L and its corresponding authentication path Auth . XM-VRF employs this sub-procedure in algorithm **VRF.Verify**.

3.1 Algorithm

$\text{ParamGen}(1^\lambda) \rightarrow \text{pp}_{\text{XM-VRF}}$: On input security parameter λ , a trusted authority produces the public parameter $\text{pp}_{\text{XM-VRF}}$ in the following way:

Algorithm 6: WOTS⁺PKfromSign

Input: Public parameter $\text{pp}_{\text{XM-VRF}} = (\text{ctr}, m, d, h, \text{ac}, w, \mathcal{F}_\lambda, \text{H}, \text{H}_1, G)$, message M , signature σ , X, k

Output: WOTS⁺ public key pk

1. Compute $l_1 = \left\lceil \frac{m}{\log w} \right\rceil$, $l_2 = \left\lfloor \frac{\log(l_1(w-1))}{\log w} \right\rfloor + 1$ and $l = l_1 + l_2$;
2. Compute $h' = \frac{h}{d}$ and $\text{ctr}' = (\text{ctr} - 1) \bmod 2^{h'}$;
3. **if** ($\text{ctr}' = 0$) **then**
4. $\text{ctr}' = 2^{h'}$;
5. **endif**
6. Set $S_t = X \in \{0, 1\}^\lambda$;
7. **for** ($p = 1; p \leq \text{ctr}'; p++$) **do**
8. Compute $G(S_t) = (f_{S_t}(0^\lambda) || f_{S_t}(1^\lambda)) = (S_t || R_t) \in \{0, 1\}^\lambda$;
9. **endfor**
10. **for** ($j = 1; j \leq w - 1; j++$) **do**
11. Compute $G(R_t) = (f_{R_t}(0^\lambda) || f_{R_t}(1^\lambda)) = (R_t || r_j) \in \{0, 1\}^\lambda$;
12. **endfor**
13. Set $r = r_{1,w-1} = (r_1, r_2, \dots, r_{w-1})$;
14. Compute $b_1, b_2, \dots, b_l$ using base w representation $M_1 \dots M_{l_1}$ of M and
15. calculating base w
16. representation $C_1 C_2 \dots C_{l_2}$ of $C = \sum_{i=1}^{l_1} (w - 1 - M_i)$ exactly in the same
17. manner as in algorithm **Sign**;
18. **for** ($i = 1; i \leq l; i++$) **do**
19. Set $\text{pk}_i = c_k^{w-1-b_i}(\sigma_i, r_{b_i+1,w-1})$ where $r_{b_i+1,w-1} = (r_{b_i+1}, r_{b_i+2},$
20. $\dots, r_{w-1})$ and $c_k(x, s)$ is presented in Fig.1;
21. **endfor**
21. **return** $\text{pk} = \{(r, k), (\text{pk}_1, \dots, \text{pk}_l)\}$;

- i. Choose a collision resistant pseudorandom function family $\mathcal{F}_\lambda = \{f_k : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda | k \in \{0, 1\}^\lambda\}$, a second preimage resistant hash function $\text{H} : \{0, 1\}^{2\lambda} \rightarrow \{0, 1\}^\lambda$ and a hash function $\text{H}_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$.
- ii. Pick $m, h \in \mathbb{N}$ to set input length and the total height of the tree respectively.
- iii. Pick $d \in \mathbb{N}$ so that d divides h to set number of layers.
- iv. Compute XMSS tree height $h' = \frac{h}{d} \in \mathbb{N}$, choose AuthCalc [12] parameter $\text{ac} \in \mathbb{N}$ such that $\text{ac} < h'$, $h' - \text{ac}$ is even and $3h' - \text{ac} \geq 4$ as well as $h' - \text{ac} \geq 2d - 2$. Pick the parameter $w \in \mathbb{N}, w > 1$.
- v. Choose a stateful forward secure PRG $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$
- vi. Initialize $\text{ctr} = 1$.
- vii. Set $\text{pp}_{\text{XM-VRF}} = (\text{ctr}, m, d, h, \text{ac}, w, \mathcal{F}_\lambda, \text{H}, \text{H}_1, G)$.

KeyGen($\text{pp}_{\text{XM-VRF}}$) $\rightarrow$ ($\text{sk}_{\text{XM-VRF}}, \text{vk}_{\text{XM-VRF}}$): A user takes input public parameter $\text{pp}_{\text{XM-VRF}}$ and outputs key pair ($\text{sk}_{\text{XM-VRF}}, \text{vk}_{\text{XM-VRF}}$) in the following manner:

- i. Choose $X \in \{0, 1\}^\lambda$ and
 Pick $S_{\text{cur},i} \xleftarrow{\$} \{0, 1\}^\lambda$ for $i = 0, 1, \dots, d - 1$.
 Pick $S_{\text{next},i} \xleftarrow{\$} \{0, 1\}^\lambda$ for $i = 0, 1, \dots, d - 2$.
 Set $S_{\text{next},d-1} = S_{\text{cur},d-1}$

- ii. Compute $l_1 = \left\lceil \frac{m}{\log w} \right\rceil$, $l_2 = \left\lfloor \frac{\log(l_1(w-1))}{\log w} \right\rfloor + 1$ and $l = l_1 + l_2$.
- iii. Compute $h' = \frac{h}{d}$, $ct = h' + \lceil \log l \rceil$ and pick bitmasks $B_i \in \{0, 1\}^{2\lambda}$ randomly for $i = 1, \dots, ct$
- iv. **for** ($i = 0; i < d; i++$) **do**
 - Set $S_t = X$
 - for** ($p = 1; p \leq 2^{h'}; p++$) **do**
 - Compute $G(S_t) = (f_{S_t}(0^\lambda) || f_{S_t}(1^\lambda)) = (S_t || R_t) \in \{0, 1\}^{2\lambda}$
 - for** ($j = 1; j \leq w - 1; j++$) **do**
 - Compute $G(R_t) = (f_{R_t}(0^\lambda) || f_{R_t}(1^\lambda)) = (R_t || r_j) \in \{0, 1\}^{2\lambda}$
 - endfor**
 - Set $r[i][p] = (r_1, r_2, \dots, r_{w-1})$
 - Set $aux[i][p] = (r[i][p], k)$
 - endfor**
- v. Compute $2^{h'}$ WOTS⁺ secret and public keys of the first tree of each layer and push public keys into Stack_i to compute the trees in the following manner:
 - Select a key $k \in \{0, 1\}^\lambda$ randomly for the key in pseudorandom function family $\mathcal{F}_\lambda = \{f_k : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda | k \in \{0, 1\}^\lambda\}$.
 - for** ($i = 0; i < d; i++$) **do**
 - Initialize empty stack Stack_i
 - Set $S_t = S_{\text{cur}, i}$
 - for** ($p = 1; p \leq 2^{h'}; p++$) **do**
 - Compute $G(S_t) = (f_{S_t}(0^\lambda) || f_{S_t}(1^\lambda)) = (S_t || R_t) \in \{0, 1\}^{2\lambda}$
 - for** ($j = 1; j \leq l; j++$) **do**
 - $sk_j[i][p] = f_{R_t}(\text{bin}(j-1))$ where $\text{bin}(j-1)$ is binary representation of $j-1$
 - $pk_j[i][p] = c_k^{w-1}(sk_j[i][p], r[i][p])$ where c_k^{w-1} is the chaining function provided in **Fig. 1** of section 2.1
 - endfor**
 - $pk[i][p] = \{aux[i][p], (pk_1[i][p], \dots, pk_l[i][p])\}$
 - $\text{TreeHash}(\text{Stack}_i, pk[i][p])$
 - endfor**
 - Initialize empty stack $\text{Stack}_i^{\text{next}}, \text{Stack}_i^{\text{SK}}$
 - endfor**
 - vi. Storing state informations $\text{St}_{\text{cur}, i}$ from stacks Stack_i for AuthCalc in the following way:
 - Denote XMSS tree nodes as $N_h[a][i](j)$, $h = 0, \dots, h'-1$ and $j = 0, \dots, 2^{h-h'} - 1$, $i = 0, 1, \dots, d-1$, $a = \text{cur}$ (current tree), next (next tree)
 - for** ($i = 0; i < d; i++$) **do**
 - Initialize state $\text{St}_{\text{cur}, i}$
 - Set $\text{St}_{\text{cur}, i} = \{\text{Store}_j[\text{cur}][i]|_{j=0}^{h'-2}, \text{Auth}_j[\text{cur}][i]|_{j=0}^{h'-1}, \text{Cache}_j[\text{cur}][i]|_{j=h'-ac}^{h'-2}, \text{Stock}_j[\text{cur}][i]|_{j=0}^{h'-3}\}$
 - Initialize $\text{Auth}_j[\text{cur}][i] = N_j[\text{cur}][i](1)$
 - $\text{Cache}_j[\text{cur}][i] = N_j[\text{cur}][i](2u+3), u = 2^{h'-j-1} - 2, \dots, 0$
 - $\text{Stock}_j[\text{cur}][i] = N_j[\text{cur}][i](3)$

- Initialize state $\text{St}_{\text{next},i}$
endfor
vii. **for** $(i = 0; i < d - 1; i++)$ **do**
 Extract $\text{rt}^{(i)} = \text{XMSS Tree}(\text{pp}_{\text{XMSS}} = (\text{pp}_{\text{WOTS}^+} = (m, w, l_1, l_2, \{0, 1\}^\lambda, \mathcal{F}_\lambda), k, h, G, H), \text{pk}[i][1], \text{pk}[i][2], \dots, \text{pk}[i][2^{h'}], B_1, B_2, \dots, B_{\text{ct}})$ from Stack_i
 $\sigma_{i+1} = \text{WOTS}^+. \text{Sign}(\text{pp}_{\text{WOTS}^+} = (m, w, l_1, l_2, \{0, 1\}^\lambda, \mathcal{F}_\lambda), \text{sk}[i+1][1], \text{rt}^{(i)})$
endfor
viii. Extract $\text{rt}^{(d-1)} = \text{XMSS Tree}(\text{pp}_{\text{XMSS}} = (\text{pp}_{\text{WOTS}^+} = (m, w, l_1, l_2, \{0, 1\}^\lambda, \mathcal{F}_\lambda), k, h, G, H), \text{pk}[d-1][1], \text{pk}[d-1][2], \dots, \text{pk}[d-1][2^{h'}], B_1, B_2, \dots, B_{\text{ct}})$ from Stack_{d-1}
Delete $\text{sk}[i][p]$ and $\text{pk}[i][p]$ for $i = 0, 1, \dots, d-1, p = 1, 2, \dots, 2^{h'}$
ix. Set $\text{sk}_{\text{XM-VRF}} = (\{S_{\text{cur},i}\}_{i=0}^{d-1}, \{S_{\text{next},i}\}_{i=0}^{d-1}, \{\sigma_i\}_{i=1}^{d-1}, \{\text{St}_{\text{cur},i}\}_{i=0}^{d-1}, \{\text{Stack}_i\}_{i=0}^{d-1}, \{\text{Stack}_i^{\text{next}}\}_{i=0}^{d-1}, \{\text{St}_{\text{next},i}\}_{i=0}^{d-1}, \{\text{Stack}_i^{\text{SK}}\}_{i=0}^{d-1}, k)$ and
 $\text{vk}_{\text{XM-VRF}} = (X, k, \{B_i\}_{i=1}^{\text{ct}}, \text{rt}^{(d-1)})$.
Output $(\text{sk}_{\text{XM-VRF}}, \text{vk}_{\text{XM-VRF}})$.

$\text{Eval}(\text{pp}_{\text{XM-VRF}}, \text{sk}_{\text{XM-VRF}}, x) \rightarrow (y_{\text{XM-VRF}}, \pi_{\text{XM-VRF}})$: The evaluator employs Eval algorithm on input $\text{pp}_{\text{XM-VRF}}, \text{sk}_{\text{XM-VRF}}$, a message $x \in \{0, 1\}^m$ to produce the evaluated pseudorandom value $y_{\text{XM-VRF}}$ and the corresponding proof $\pi_{\text{XM-VRF}}$ of correct computation of $y_{\text{XM-VRF}}$ by approach detailed below:

- i. Compute $h' = \frac{h}{d}$ and $\text{ctr}' = \text{ctr} \bmod 2^{h'}$ by extracting h, d, ctr from $\text{pp}_{\text{XM-VRF}}$
If $\text{ctr}' = 0$, then $\text{ctr}' = 2^{h'}$
Set $S_t = S_{\text{cur},0}$
for $(i = 0; i < \text{ctr}'; i++)$ **do**
 Compute $G(S_t) = (f_{S_t}(0^\lambda) || f_{S_t}(1^\lambda)) = (S_t || R_t) \in \{0, 1\}^{2\lambda}$
endfor
Compute $l_1 = \left\lceil \frac{m}{\log w} \right\rceil$, $l_2 = \left\lfloor \frac{\log(l_1(w-1))}{\log w} \right\rfloor + 1$ and $l = l_1 + l_2$
Compute $\text{sk}_j = f_{R_t}(\text{bin}(j-1))$ for $j = 1, 2, \dots, l$ where $\text{bin}(j-1)$ is binary representation of $j-1$
Set $S_t = X$
for $(p = 1; p \leq \text{ctr}'; p++)$ **do**
 Compute $G(S_t) = (f_{S_t}(0^\lambda) || f_{S_t}(1^\lambda)) = (S_t || R_t) \in \{0, 1\}^{2\lambda}$
endfor
for $(j = 1; j \leq w-1; j++)$ **do**
 Compute $G(R_t) = (f_{R_t}(0^\lambda) || f_{R_t}(1^\lambda)) = (R_t || r_j) \in \{0, 1\}^{2\lambda}$
endfor
Set $\text{aux} = (r, k)$ where $r = (r_1, r_2, \dots, r_{w-1})$ and k is extracted from $\text{sk}_{\text{XM-VRF}}$
Set $\text{sk} = \{\text{aux}, (\text{sk}_1, \dots, \text{sk}_l)\}$
ii. Compute $\sigma_0 \leftarrow \text{WOTS}^+. \text{Sign}(\text{pp}_{\text{WOTS}^+} = (m, w, l_1, l_2, \{0, 1\}^\lambda, \mathcal{F}_\lambda), \text{sk}, x)$ where $\sigma_0 \in \{0, 1\}^{l \times \lambda}$
Extract $\text{Auth}(\text{ctr}') = \{\text{Auth}_i(\text{ctr}')\}_{i=0}^{d-1}$ from $\{\text{St}_{\text{cur},i}\}_{i=0}^{d-1}$
Extract $\sigma_1, \dots, \sigma_{d-1}$ from $\text{sk}_{\text{XM-VRF}}$
iii. Set $\Sigma = (\sigma_0, \text{Auth}_0(\text{ctr}') \dots, \sigma_{d-1}, \text{Auth}_{d-1}(\text{ctr}'))$.
iv. Compute WOTS^+ public keys of the next tree of each layer and push them into $\text{STACK}_i^{\text{next}}$ in the following manner

```

for ( $i = 0; i < d; i++$ ) do
  Set  $S_t = S_{\text{next},i}$  and  $S'_t = X$ 
  for ( $p = 1; p \leq \text{ctr}'; p++$ ) do
    Compute  $G(S_t) = (f_{S_t}(0^\lambda) || f_{S_t}(1^\lambda)) = (S_t || R_t) \in \{0, 1\}^{2\lambda}$ 
    Compute  $G(S'_t) = (f_{S'_t}(0^\lambda) || f_{S'_t}(1^\lambda)) = (S'_t || R'_t) \in \{0, 1\}^{2\lambda}$ 
  endfor
  for ( $j = 1; j \leq w - 1; j++$ ) do
    Compute  $G(R'_t) = (f_{R'_t}(0^\lambda) || f_{R'_t}(1^\lambda)) = (R'_t || r'_j) \in \{0, 1\}^{2\lambda}$ 
  endfor
  Set  $r[i][\text{ctr}'] = (r'_1, \dots, r'_{w-1})$ ,
   $\text{sk}_j[i][\text{ctr}'] = f_{R_t}(\text{bin}(j-1))$  and  $\text{pk}_j[i][\text{ctr}'] = c_k^{w-1}(\text{sk}_j[i][\text{ctr}'], r[i][\text{ctr}'])$ 
  for all  $j = 1, \dots, l$  where  $c_k^{w-1}$  is the chaining function defined in
  Fig. 1
   $\text{aux}[i][\text{ctr}'] = (r[i][\text{ctr}'], k)$ 
   $\text{pk}[i][\text{ctr}'] = \{\text{aux}[i][\text{ctr}'], (\text{pk}_1[i][\text{ctr}'], \dots, \text{pk}_l[i][\text{ctr}'])\}$ 
  Run  $\text{TreeHash}(\text{Stack}_i^{\text{next}}, \text{pk}[i][\text{ctr}'])$ 
  if ( $\text{ctr}' = 1$ ) then
     $\text{sk}[i][\text{ctr}'] = \{\text{aux}[i][\text{ctr}'], (\text{sk}_1[i][\text{ctr}'], \dots, \text{sk}_l[i][\text{ctr}'])\}$ 
    Run  $\text{TreeHash}(\text{Stack}_i^{\text{SK}}, \text{sk}[i][\text{ctr}'])$ 
  endif
endfor
v. Store state informations  $\text{St}_{\text{next},i}$  from stacks  $\text{STACK}_i^{\text{next}}$  for AuthCalc using
the following steps:
Denote XMSS tree nodes as  $N_h[a][i](j)$ ,  $h = 0, \dots, h'-1$  and  $j = 0, \dots, 2^{h-h'} - 1$ ,
 $i = 0, 1, \dots, d-1$ ,  $a = \text{cur}$  (current tree),  $\text{next}$  (next tree)
for ( $i = 0; i < d; i++$ ) do
  Set  $\text{St}_{\text{next},i} = \{\text{Store}_j[\text{next}][i]_{j=0}^{h'-2}, \text{Auth}_j[\text{next}][i]_{j=0}^{h'-1}, \text{Cache}_j[\text{next}][i]_{j=h'-ac}^{h'-2},$ 
     $\text{Stock}_j[\text{next}][i]_{j=0}^{h'-3}\}$ 
  Initialize  $\text{Auth}_j[\text{next}][i] = N_j[\text{next}][i](1)$ 
   $\text{Cache}_j[\text{next}][i] = N_j[\text{next}][i](2u+3)$ ,  $u = 2^{h'-j-1} - 2, \dots, 0$ 
   $\text{Stock}_j[\text{next}][i] = N_j[\text{next}][i](3)$ 
endfor
vi. if ( $\text{ctr}' = 2^{h'}$ ) then
  for ( $i = 0; i < d-1; i++$ ) do
    Extract  $\{0, 1\}^\lambda \ni \text{rt}^{(i)} \leftarrow \text{XMSS Tree}(\text{pp}_{\text{XMSS}} = (\text{pp}_{\text{WOTS}^+} = (m, w, l_1, l_2,$ 
     $\{0, 1\}^\lambda, \mathcal{F}_\lambda), k, h, G, H), \text{pk}[i][1], \text{pk}[i][2], \dots, \text{pk}[i][2^{h'}], B_1, B_2, \dots, B_{\text{ct}})$ 
    from  $\text{Stack}_i^{\text{next}}$ 
    Fetch  $\text{sk}[i+1][1]$  from  $\text{STACK}_i^{\text{SK}}$ 
     $\sigma'_{i+1} = \text{WOTS}^+. \text{Sign}(\text{pp}_{\text{WOTS}^+}, \text{sk}[i+1][1], \text{rt}^{(i)})$ 
  endfor
endif
vii. for ( $i = 0; i < d; i++$ ) do
  Run  $\text{AuthCalc}(\text{ctr}', h, ac, \text{St}_{\text{cur},i}, H) \rightarrow (\text{Auth}(\text{ctr}' + 1), \text{St})$ 
  Set  $\text{St}_{\text{cur},i} = \text{St}$ 

```

```

endfor
Set  $\text{ctr} = \text{ctr} + 1$ 
viii. if ( $\text{ctr}' = 1$ ) then
    Pick  $S'_{\text{next},i} \xleftarrow{\$} \{0, 1\}^\lambda$  for  $i = 0, 1, \dots, d-2$ 
endif
if ( $\text{ctr}' = 2^{h'}$ ) then
    Set  $S_{\text{cur},i} = S_{\text{next},i}$  and  $S_{\text{next},i} = S'_{\text{next},i}$  for  $i = 0, 1, \dots, d-2$ 
    for ( $i = 0; i < d; i++$ ) do
         $\text{St}_{\text{cur},i} = \text{St}_{\text{next},i}$ 
        Clear  $\text{Stack}_i^{\text{next}}, \text{Stack}_i^{\text{SK}}, \text{St}_{\text{next},i}$ 
    endfor
    Set  $\sigma_i = \sigma'_i$  for  $i = 0, 1, \dots, d-1$ 
endif
ix. Set  $\pi_{\text{XM-VRF}} = \Sigma$  and compute  $y_{\text{XM-VRF}} = H_1(\Sigma, x) \in \{0, 1\}^\lambda$ 
    Output  $(y_{\text{XM-VRF}}, \pi_{\text{XM-VRF}})$ 

```

Verify($\text{pp}_{\text{XM-VRF}}, \text{vk}_{\text{XM-VRF}}, x, y_{\text{XM-VRF}}, \pi_{\text{XM-VRF}}$) $\rightarrow$ 1/0: The verifier checks the validity of the pseudorandom value $y_{\text{XM-VRF}}$ on x using the verification key $\text{vk}_{\text{XM-VRF}}$ and the proof $\pi_{\text{XM-VRF}}$ with the subsequent syntax:

```

i. Parse  $\pi_{\text{XM-VRF}} = (\sigma_0, \text{Auth}_0, \sigma_1, \text{Auth}_1, \dots, \sigma_{d-1}, \text{Auth}_{d-1})$ 
    Compute  $l_1 = \left\lceil \frac{m}{\log w} \right\rceil$ ,  $l_2 = \left\lceil \frac{\log(l_1(w-1))}{\log w} \right\rceil + 1$  and  $l = l_1 + l_2$ 
    Extract  $X$  from  $\text{vk}_{\text{XM-VRF}}$ 
ii. Run  $\text{WOTS}^+ \text{PKfromSign}(\text{pp}_{\text{WOTS}^+}, x, \sigma_0, X, k) \rightarrow \text{pk}^{(0)}$ 
    Parse  $\text{pk}^{(0)} = \{(r, k), (\text{pk}_1, \dots, \text{pk}_l)\}$ 
    Now compute  $\text{L-Tree}(\text{pp}_{\text{WOTS}^+}, (\text{pk}_1, \dots, \text{pk}_l), (B_1, \dots, B_{\lceil \log l \rceil}) \rightarrow L_0$ 
    Run  $\text{Auth2Root}(\text{pp}_{\text{XM-VRF}}, \text{Auth}_0, L_0, B_{\lceil \log l \rceil+1}, \dots, B_{\text{ct}}) \rightarrow x_1$ 
iii. for ( $j = 1; j \leq d-1; j++$ ) do
    Run  $\text{WOTS}^+ \text{PKfromSign}(\text{pp}_{\text{WOTS}^+}, x_j, \sigma_j, X, k) \rightarrow \text{pk}^{(j)}$ 
    Parse  $\text{pk}^{(j)} = \{(r, k), (\text{pk}_1, \dots, \text{pk}_l)\}$ 
    Now compute  $\text{L-Tree}(\text{pp}_{\text{WOTS}^+}, (\text{pk}_1, \dots, \text{pk}_l), (B_1, \dots, B_{\lceil \log l \rceil}) \rightarrow L_j$ 
    Run  $\text{Auth2Root}(\text{pp}_{\text{XM-VRF}}, \text{Auth}_j, L_j, B_{\lceil \log l \rceil+1}, \dots, B_{\text{ct}}) \rightarrow x_{j+1}$ 
endfor
iv. Extract  $\text{rt}^{(d-1)}$  from  $\text{vk}_{\text{XM-VRF}}$ 
    if ( $x_d = \text{rt}^{(d-1)}$ ) then
        proceed to next step
    else
        output 0
    endif
v. if ( $H_1(\pi_{\text{XM-VRF}}, x) = y_{\text{XM-VRF}}$ ) then
    output 1
else
    output 0
endif

```

3.2 New Algorithm

ParamGen(1^λ) $\rightarrow$ $\text{pp}_{\text{XM-VRF}}$: On input security parameter λ , a trusted authority produces the public parameter $\text{pp}_{\text{XM-VRF}}$ in the following way:

- i. Choose a collision resistant pseudorandom function family $\mathcal{F}_\lambda = \{f_k : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda | k \in \{0, 1\}^\lambda\}$, a second preimage resistant hash function $H : \{0, 1\}^{2\lambda} \rightarrow \{0, 1\}^\lambda$ and a hash function $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$.
- ii. Pick $m, h \in \mathbb{N}$ to set input length and the total height of the tree respectively.
- iii. Pick $d \in \mathbb{N}$ so that d divides h to compute height of XMSS trees in each layer.
- iv. Compute XMSS tree height $h' = \frac{h}{d} \in \mathbb{N}$, choose AuthCalc [12] parameter $\text{ac} \in \mathbb{N}$ such that $\text{ac} < h', h' - \text{ac}$ is even and $3h' - \text{ac} \geq 4$ as well as $h' - \text{ac} \geq 2d - 2$. Pick the parameter $w \in \mathbb{N}, w > 1$.
- v. Choose a stateful forward secure PRG $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$
- vi. Initialize $\text{ctr} = 1$.
- vii. Set $\text{pp}_{\text{XM-VRF}} = (\text{ctr}, m, d, h, \text{ac}, w, \mathcal{F}_\lambda, H, H_1, G)$.

KeyGen($\text{pp}_{\text{XM-VRF}}$) $\rightarrow$ ($\text{sk}_{\text{XM-VRF}}, \text{vk}_{\text{XM-VRF}}$): A user takes input public parameter $\text{pp}_{\text{XM-VRF}}$ and outputs key pair ($\text{sk}_{\text{XM-VRF}}, \text{vk}_{\text{XM-VRF}}$) in the following manner:

- i. Choose $X_1 \in \{0, 1\}^\lambda, X_2 \in \{0, 1\}^\lambda$ and
 Pick $S_{\text{cur}, i} \xleftarrow{\$} \{0, 1\}^\lambda$ for $i = 1, 2$
 Pick $S_{\text{next}} \xleftarrow{\$} \{0, 1\}^\lambda$
- ii. Compute $l_1 = \left\lceil \frac{m}{\log w} \right\rceil, l_2 = \left\lceil \frac{\log(l_1(w-1))}{\log w} \right\rceil + 1$ and $l = l_1 + l_2$.
- iii. Compute $h' = \frac{h}{d}, \text{ct} = h' + \lceil \log l \rceil$ and pick bitmasks $B_i \in \{0, 1\}^{2\lambda}$ randomly for $i = 1, \dots, \text{ct}$
- iv. **for** ($i = 1; i \leq 2; i++$) **do**
 Set $S_t = X_i$
for ($p = 1; p \leq 2^{h'}; p++$) **do**
 Compute $G(S_t) = (f_{S_t}(0^\lambda) || f_{S_t}(1^\lambda)) = (S_t || R_t) \in \{0, 1\}^{2\lambda}$
for ($j = 1; j \leq w - 1; j++$) **do**
 Compute $G(R_t) = (f_{R_t}(0^\lambda) || f_{R_t}(1^\lambda)) = (R_t || r_j) \in \{0, 1\}^{2\lambda}$
endfor
 Set $r[i][p] = (r_1, r_2, \dots, r_{w-1})$
endfor
- v. Compute $2^{h'}$ WOTS⁺ secret and public keys of the first tree of each layer and push public keys into Stack_i to compute the trees in the following manner:
 Select a key $k \in \{0, 1\}^\lambda$ randomly for the key in pseudorandom function family $\mathcal{F}_\lambda = \{f_k : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda | k \in \{0, 1\}^\lambda\}$.
 Initialize empty stacks $\text{Stack}_1, \text{Stack}_2, \text{Stack}^{\text{next}}$
for ($i = 1; i \leq 2; i++$) **do**
 Set $S_t = S_{\text{cur}, i}$
for ($p = 1; p \leq 2^{h'}; p++$) **do**

- Set $\text{aux}[i][p] = (r[i][p], k)$
 Compute $G(S_t) = (f_{S_t}(0^\lambda) || f_{S_t}(1^\lambda)) = (S_t || R_t) \in \{0, 1\}^{2\lambda}$
for $(j = 1; j \leq l; j++)$ **do**
 $\text{sk}_j[i][p] = f_{R_t}(\text{bin}(j-1))$ where $\text{bin}(j-1)$ is binary representation of $j-1$
 $\text{pk}_j[i][p] = c_k^{w-1}(\text{sk}_j[i][p], r[i][p])$ where c_k^{w-1} is the chaining function provided in **Fig. 1** of section 2.1
endfor
 $\text{sk}[i][p] = \{\text{aux}[i][p], (\text{sk}_1[i][p], \dots, \text{sk}_l[i][p])\}$
 $\text{pk}[i][p] = \{\text{aux}[i][p], (\text{pk}_1[i][p], \dots, \text{pk}_l[i][p])\}$
 TreeHash(Stack_i, pk[i][p])
endfor
endfor
- vi. Storing state informations $\text{St}_{\text{cur},i}$ from stacks Stack_i for AuthCalc in the following way:
 Denote XMSS tree nodes as $N_h[a][1](j), N_h[2](j), h = 0, \dots, h' - 1$ and $j = 0, \dots, 2^{h-h'} - 1, a = \text{cur}$ (current tree), next (next tree)
for $(i = 1; i \leq 2; i++)$ **do**
 Initialize state $\text{St}_{\text{cur},i}$
 Set $\text{St}_{\text{cur},i} = \{\text{Store}_j[\text{cur}][i]|_{j=0}^{h'-2}, \text{Auth}_j[\text{cur}][i]|_{j=0}^{h'-1}, \text{Cache}_j[\text{cur}][i]|_{j=h'-ac}^{h'-2}, \text{Stock}_j[\text{cur}][i]|_{j=0}^{h'-3}\}$
 Initialize $\text{Auth}_j[\text{cur}][i] = N_j[\text{cur}][i](1)$
 $\text{Cache}_j[\text{cur}][i] = N_j[\text{cur}][i](2u+3), u = 2^{h'-j-1} - 2, \dots, 0$
 $\text{Stock}_j[\text{cur}][i] = N_j[\text{cur}][i](3)$
endfor
 Initialize state St_{next}
- vii. Extract $\text{rt}^{(1)} = \text{XMSS Tree}(\text{pp}_{\text{XMSS}} = (\text{pp}_{\text{WOTS}^+} = (m, w, l_1, l_2, \{0, 1\}^\lambda, \mathcal{F}_\lambda), k, h, G, H), \text{pk}[1][1], \text{pk}[1][2], \dots, \text{pk}[1][2^{h'}], B_1, B_2, \dots, B_{\text{ct}})$ from Stack₁
 Compute $\sigma_2 = \text{WOTS}^+. \text{Sign}(\text{pp}_{\text{WOTS}^+} = (m, w, l_1, l_2, \{0, 1\}^\lambda, \mathcal{F}_\lambda), \text{sk}[2][1], \text{rt}^{(1)})$
- viii. Extract $\text{rt}^{(2)} = \text{XMSS Tree}(\text{pp}_{\text{XMSS}} = (\text{pp}_{\text{WOTS}^+} = (m, w, l_1, l_2, \{0, 1\}^\lambda, \mathcal{F}_\lambda), k, h, G, H), \text{pk}[2][1], \text{pk}[2][2], \dots, \text{pk}[2][2^{h'}], B_1, B_2, \dots, B_{\text{ct}})$ from Stack₂
 Delete $\text{sk}[i][p]$ and $\text{pk}[i][p]$ for $i = 1, 2, p = 1, 2, \dots, 2^{h'}$
- ix. Set $\text{sk}_{\text{XM-VRF}} = (S_{\text{cur},1}, S_{\text{cur},2}, S_{\text{next}}, \sigma_1, \text{St}_{\text{cur},1}, \text{St}_{\text{cur},2}, \text{St}_{\text{next}}, \text{Stack}_1, \text{Stack}_2, \text{Stack}^{\text{next}}, X_1, X_2, k)$ and
 $\text{vk}_{\text{XM-VRF}} = (X_1, X_2, k, \{B_i\}_{i=1}^{\text{ct}}, \text{rt}^{(2)})$
 Output $(\text{sk}_{\text{XM-VRF}}, \text{vk}_{\text{XM-VRF}})$.

$\text{Eval}(\text{pp}_{\text{XM-VRF}}, \text{sk}_{\text{XM-VRF}}, x) \rightarrow (y_{\text{XM-VRF}}, \pi_{\text{XM-VRF}})$: The evaluator employs Eval algorithm on input $\text{pp}_{\text{XM-VRF}}, \text{sk}_{\text{XM-VRF}}$, a message $x \in \{0, 1\}^m$ to produce the evaluated pseudorandom value $y_{\text{XM-VRF}}$ and the corresponding proof $\pi_{\text{XM-VRF}}$ of correct computation of $y_{\text{XM-VRF}}$ and update the secret key $\text{sk}_{\text{XM-VRF}}$ by approach detailed below:

- i. Compute $h' = \frac{h}{d}$ and $\text{ctr}' = \text{ctr} \bmod 2^{h'}$ by extracting h, d, ctr from $\text{pp}_{\text{XM-VRF}}$
 If $\text{ctr}' = 0$, then $\text{ctr}' = 2^{h'}$

Compute $\text{ctr}'' = \left\lceil \frac{\text{ctr}}{2^{h'}} \right\rceil$
 If $\text{ctr}'' > 2^{h'}$, then $\text{ctr}'' = \text{ctr}'' \bmod 2^{h'}$
 Set $S_t = S_{\text{cur},1}$
for $(i = 0; i < \text{ctr}'; i++)$ **do**
 Compute $G(S_t) = (f_{S_t}(0^\lambda) || f_{S_t}(1^\lambda)) = (S_t || R_t) \in \{0, 1\}^{2\lambda}$
endfor
 Compute $l_1 = \left\lceil \frac{m}{\log w} \right\rceil$, $l_2 = \left\lfloor \frac{\log(l_1(w-1))}{\log w} \right\rfloor + 1$ and $l = l_1 + l_2$
 Compute $\text{sk}_j = f_{R_t}(\text{bin}(j-1))$ for $j = 1, 2, \dots, l$ where $\text{bin}(j-1)$ is binary representation of $j-1$
 Set $S_t = X_1$
for $(p = 1; p \leq \text{ctr}'; p++)$ **do**
 Compute $G(S_t) = (f_{S_t}(0^\lambda) || f_{S_t}(1^\lambda)) = (S_t || R_t) \in \{0, 1\}^{2\lambda}$
endfor
for $(j = 1; j \leq w-1; j++)$ **do**
 Compute $G(R_t) = (f_{R_t}(0^\lambda) || f_{R_t}(1^\lambda)) = (R_t || r_j) \in \{0, 1\}^{2\lambda}$
endfor
 Set $\text{aux} = (r, k)$ where $r = (r_1, r_2, \dots, r_{w-1})$ and k is extracted from $\text{sk}_{\text{XM-VRF}}$
 Set $\text{sk} = \{\text{aux}, (\text{sk}_1, \dots, \text{sk}_l)\}$
 ii. Compute $\sigma_1 \leftarrow \text{WOTS}^+.\text{Sign}(\text{pp}_{\text{WOTS}^+} = (m, w, l_1, l_2, \{0, 1\}^\lambda, \mathcal{F}_\lambda), \text{sk}, x)$ where $\sigma_1 \in \{0, 1\}^{l \times \lambda}$
 Extract $\text{Auth}(\text{ctr}') = \{\text{Auth}_1(\text{ctr}'), \text{Auth}_2(\text{ctr}')\}$ from $\text{St}_{\text{cur},1}, \text{St}_{\text{cur},2}$
 Extract σ_2 from $\text{sk}_{\text{XM-VRF}}$
 iii. Set $\Sigma = (\sigma_1, \text{Auth}_1(\text{ctr}'), \sigma_2, \text{Auth}_2(\text{ctr}'))$.
 iv. Compute WOTS^+ public keys of the next tree of lower layer and push them into $\text{Stack}^{\text{next}}$ in the following manner
 Set $S_t = S_{\text{next}}$ and $S'_t = X_1$
 for $(p = 1; p \leq \text{ctr}'; p++)$ **do**
 Compute $G(S_t) = (f_{S_t}(0^\lambda) || f_{S_t}(1^\lambda)) = (S_t || R_t) \in \{0, 1\}^{2\lambda}$
 Compute $G(S'_t) = (f_{S'_t}(0^\lambda) || f_{S'_t}(1^\lambda)) = (S'_t || R'_t) \in \{0, 1\}^{2\lambda}$
 endfor
 for $(j = 1; j \leq w-1; j++)$ **do**
 Compute $G(R'_t) = (f_{R'_t}(0^\lambda) || f_{R'_t}(1^\lambda)) = (R'_t || r'_j) \in \{0, 1\}^{2\lambda}$
 endfor
 Set $r[[1][\text{ctr}']] = (r'_1, \dots, r'_{w-1})$,
 $\text{sk}_j[1][\text{ctr}'] = f_{R_t}(\text{bin}(j-1))$ and $\text{pk}_j[1][\text{ctr}'] = c_k^{w-1}(\text{sk}_j[1][\text{ctr}'], r[1][\text{ctr}'])$
 for all $j = 1, \dots, l$ where c_k^{w-1} is the chaining function defined in **Fig. 1**
 $\text{aux}[1][\text{ctr}'] = (r[1][\text{ctr}'], k)$
 $\text{pk}[1][\text{ctr}'] = \{\text{aux}[1][\text{ctr}'], (\text{pk}_1[1][\text{ctr}'], \dots, \text{pk}_l[1][\text{ctr}'])\}$
 Run $\text{TreeHash}(\text{Stack}^{\text{next}}, \text{pk}[1][\text{ctr}'])$
 v. Store state informations St_{next} from stacks $\text{STACK}^{\text{next}}$ for AuthCalc using the following steps:
 Denote XMSS tree nodes as $N_h[a][1](j), N_h[2](j), h = 0, \dots, h' - 1$ and $j = 0, \dots, 2^{h-h'} - 1, a = \text{cur}$ (current tree), next (next tree)
 for $(i = 1; i \leq 2; i++)$ **do**
 Set $\text{St}_{\text{next}} = \{\text{Store}_j[\text{next}][i]_{j=0}^{h'-2}, \text{Auth}_j[\text{next}][i]_{j=0}^{h'-1}, \text{Cache}_j[\text{next}][i]_{j=h'-ac}^{h'-2},$

```

    Stockj[next][i]  $\leftarrow \{0, 1\}^{h'-3}$ 
    Initialize Authj[next][i] = Nj[next][i](1)
    Cachej[next][i] = Nj[next][i](2u + 3), u = 2h'-j-1 - 2, ..., 0
    Stockj[next][i] = Nj[next][i](3)
  endfor
vi. if (ctr' = 2h') then
  Extract {0, 1}λ ∋ rt(1) ← XMSS Tree(ppXMSS = (ppWOTS+ = (m, w, l1, l2, {0, 1}λ,
  Fλ), k, h, G, H), pk[i][1], pk[i][2], ..., pk[i][2h'], B1, B2, ..., Bct) from Stacknext
  Set St = Scur,2
  for (i = 0; i < ctr'' + 1; i++) do
    Compute G(St) = (fSt(0λ) || fSt(1λ)) = (St || Rt) ∈ {0, 1}2λ
  endfor
  Compute skj = fRt(bin(j - 1)) for j = 1, 2, ..., l where bin(j - 1) is binary
  representation of j - 1
  Set St = X2
  for (p = 1; p ≤ ctr'' + 1; p++) do
    Compute G(St) = (fSt(0λ) || fSt(1λ)) = (St || Rt) ∈ {0, 1}2λ
  endfor
  for (j = 1; j ≤ w - 1; j++) do
    Compute G(Rt) = (fRt(0λ) || fRt(1λ)) = (Rt || rj) ∈ {0, 1}2λ
  endfor
  Set aux = (r, k) where r = (r1, r2, ..., rw-1) and k is extracted from
  skXM-VRF
  Set sk' = {aux, (sk1, ..., skl)}
  σ'2 = WOTS+.Sign(ppWOTS+, sk', rt(1))
endif
vii. for (i = 1; i ≤ 2; i++) do
  Run AuthCalc(ctr', h, ac, Stcur,i, H) → (Auth(ctr' + 1), St)
  Set Stcur,i = St
endfor
Set ctr = ctr + 1
viii. if (ctr' = 1) then
  Pick S'next  $\xleftarrow{\$}$  {0, 1}λ
endif
if (ctr' = 2h') then
  Set Scur,1 = Snext and Snext = S'next
  Stcur,1 = Stnext
  Clear Stacknext, Stnext
  Set σ2 = σ'2
endif
ix. Set πXM-VRF = Σ and compute yXM-VRF = H1(Σ, x) ∈ {0, 1}λ
  Output (yXM-VRF, πXM-VRF)

```

Verify(pp_{XM-VRF}, vk_{XM-VRF}, x, y_{XM-VRF}, π_{XM-VRF}) → 1/0: The verifier checks the validity of the pseudorandom value y_{XM-VRF} on x using the verification key vk_{XM-VRF} and the proof π_{XM-VRF} with the subsequent syntax:

- i. Parse $\pi_{\text{XM-VRF}} = (\sigma_1, \text{Auth}_1, \sigma_2, \text{Auth}_2)$
 Compute $l_1 = \left\lceil \frac{m}{\log w} \right\rceil$, $l_2 = \left\lceil \frac{\log(l_1(w-1))}{\log w} \right\rceil + 1$ and $l = l_1 + l_2$
 Extract X_1 from $\text{vk}_{\text{XM-VRF}}$
- ii. Run $\text{WOTS}^+ \text{PKfromSign}(\text{pp}_{\text{WOTS}^+}, x, \sigma_1, X_1, k) \rightarrow \text{pk}^{(1)}$
 Parse $\text{pk}^{(1)} = \{(r, k), (\text{pk}_1, \dots, \text{pk}_l)\}$
 Now compute $\text{L-Tree}(\text{pp}_{\text{WOTS}^+}, (\text{pk}_1, \dots, \text{pk}_l), (B_1, \dots, B_{\lceil \log l \rceil})) \rightarrow L_1$
 Run $\text{Auth2Root}(\text{pp}_{\text{XM-VRF}}, \text{Auth}_1, L_1, B_{\lceil \log l \rceil + 1}, \dots, B_{\text{ct}}) \rightarrow x_1$
- iii. Extract X_2 from $\text{vk}_{\text{XM-VRF}}$
 Run $\text{WOTS}^+ \text{PKfromSign}(\text{pp}_{\text{WOTS}^+}, x_1, \sigma_2, X_2, k) \rightarrow \text{pk}^{(2)}$
 Parse $\text{pk}^{(2)} = \{(r, k), (\text{pk}_1, \dots, \text{pk}_l)\}$
 Now compute $\text{L-Tree}(\text{pp}_{\text{WOTS}^+}, (\text{pk}_1, \dots, \text{pk}_l), (B_1, \dots, B_{\lceil \log l \rceil})) \rightarrow L_2$
 Run $\text{Auth2Root}(\text{pp}_{\text{XM-VRF}}, \text{Auth}_2, L_2, B_{\lceil \log l \rceil + 1}, \dots, B_{\text{ct}}) \rightarrow x_2$
- iv. Extract $\text{rt}^{(2)}$ from $\text{vk}_{\text{XM-VRF}}$
if $(x_2 = \text{rt}^{(2)})$ **then**
 proceed to next step
else
 output 0
endif
- v. **if** $(H_1(\pi_{\text{XM-VRF}}, x) = y_{\text{XM-VRF}})$ **then**
 output 1
else
 output 0
endif

3.3 Storage Analysis

We present the storage requirements, in terms of the number of binary strings, for the secret key $\text{sk}_{\text{XM-VRF}}$, verification key $\text{vk}_{\text{XM-VRF}}$, and proof $\pi_{\text{XM-VRF}}$ in Table 3.

	$\text{sk}_{\text{XM-VRF}}$	$\text{vk}_{\text{XM-VRF}}$	$\pi_{\text{XM-VRF}}$
Storage	$9 \cdot 2^{\frac{h}{d}} + 6 \frac{h}{d} + l - 9$ strings from $\{0, 1\}^\lambda$	4 strings from $\{0, 1\}^\lambda$ + $\frac{h}{d} + \lceil \log l \rceil$ strings from $\{0, 1\}^{2\lambda}$	$2(l + \frac{h}{d})$ string from $\{0, 1\}^\lambda$

Table 3. Storage of the secret key $\text{sk}_{\text{XM-VRF}}$, verification key $\text{vk}_{\text{XM-VRF}}$, and proof $\pi_{\text{XM-VRF}}$

3.4 Correctness

Here we will discuss the correctness of our construction. More precisely we have to show the provability property of the above algorithm in succeeding manner.

- i. Assume $(y_{\text{XM-VRF}}, \pi_{\text{XM-VRF}})$ is the output of the $\text{VRF.Eval}(\cdot)$ where $\text{sk}_{\text{XM-VRF}}$, $\text{vk}_{\text{XM-VRF}}$ are generated honestly.

- ii. Parse $\pi_{\text{XM-VRF}} = (i, \sigma_0, \text{Auth}_0, \sigma_1, \text{Auth}_1, \dots, \sigma_{d-1}, \text{Auth}_{d-1})$. So we will have all correct WOTS^+ signatures $\sigma_0, \dots, \sigma_{d-1}$.
- iii. Therefore the corresponding WOTS^+ public key $\text{pk}^{(0)}$ can be computed from σ_0 using $\text{WOTS}^+ \text{PKfromSign}(\cdot)$ on inputs σ_0, x . Now use $\text{pk}^{(0)}$ and Auth_0 to compute the root of the lowest XMSS tree by running $\text{Auth2Root}(\cdot)$, let's say the generated root is x_1 .
- iv. **for** ($j = 1; j \leq d - 1; j++$) **do**
 Repeat same process with $\sigma_j, x_j, \text{Auth}_j$ using the algorithms $\text{WOTS}^+ \text{PKfromSign}(\cdot)$ and $\text{Auth2Root}(\cdot)$ to generate x_{j+1}
 endfor
- v. At last we will generate x_d which should be the root node of the top XMSS tree and this root is also stored in the $\text{vk}_{\text{XM-VRF}}$ as $\text{rt}^{(d-1)}$.
- vi. Since $(y_{\text{XM-VRF}}, \pi_{\text{XM-VRF}})$ is the output of the $\text{VRF.Eval}(\cdot)$ where $\text{sk}_{\text{XM-VRF}}, \text{vk}_{\text{XM-VRF}}$ are generated honestly, so the comparison check $x_d \stackrel{?}{=} \text{rt}^{(d-1)}$ and checking of $H_1(\pi_{\text{VRF}}, x) \stackrel{?}{=} y_{\text{VRF}}$ both will pass. So $\text{VRF.Verify}(\cdot)$ will output 1. Hence our algorithm satisfies provability property.

3.5 Security

In this section we analyze the uniqueness and pseudorandomness of our XM-VRF. First, we deep dive into the most critical property uniqueness. Next, we will focus on the other property pseudorandomness.

Theorem 4. *If hash function H and function family $\mathcal{F}_\lambda$ in the public parameter $\text{pp}_{\text{XM-VRF}}$ are collision resistant, then XM-VRF satisfies the uniqueness property in the random oracle model.*

Proof. We prove uniqueness of XM-VRF by reduction technique. Let $\mathcal{A}$ be a PPT adversary against the uniqueness of our XM-VRF scheme. We will show that if $\mathcal{A}$ can break the uniqueness of XM-VRF, then there is an adversary $\mathcal{B}$ which can break the uniqueness of the WOTS^+ . Suppose $\mathcal{A}$ can break the uniqueness of XM-VRF i.e, $\mathcal{A}$ have two different evaluations and corresponding proofs for the same input $(y_{\text{VRF},1}, y_{\text{VRF},2}, \pi_{\text{VRF},1}, \pi_{\text{VRF},2}, x)$ satisfying

$$\text{VRF.Verify}(\text{pp}_{\text{XM-VRF}}, \text{pk}_{\text{XM-VRF}}, x, y_{\text{VRF},1}, \pi_{\text{VRF},1}) = \text{VRF.Verify}(\text{pp}_{\text{XM-VRF}}, \text{pk}_{\text{XM-VRF}}, x, y_{\text{VRF},2}, \pi_{\text{VRF},2}) = 1$$

when $y_{\text{VRF},1} \neq y_{\text{VRF},2}$

Now recall that $y_{\text{VRF},i} = H_1(\Sigma_i, x)$ and $\pi_{\text{VRF},i} = \Sigma_i$ for $i = 1, 2$. Since $y_{\text{VRF},1} \neq y_{\text{VRF},2}$, so $\Sigma_1 \neq \Sigma_2$.

We also know that $\Sigma_j = (\sigma_0^j, \text{Auth}_0^j, \sigma_1^j, \text{Auth}_1^j, \dots, \sigma_{d-1}^j, \text{Auth}_{d-1}^j)$ for $j = 1, 2$. Since same ctr or leaf index $\text{pp}_{\text{XM-VRF}}$ is used for VRF.Eval , same tree root is used and the used hash function H is collision resistant, so $\text{Auth}_j^1 = \text{Auth}_j^2$ for $j = 1, 2, \dots, d-1$. Therefore $\Sigma_1 \neq \Sigma_2$ implies $\sigma_j^1 \neq \sigma_j^2$ for atleast one j . Now the adversary $\mathcal{B}$ returns σ_j^1, σ_j^2 to break the uniqueness of WOTS^+ . But by Theorem 1, WOTS^+ is unique if function family $\mathcal{F}_\lambda$ is collision resistant. So contradiction. Hence XM-VRF is unique. $\square$

Theorem 5. XM-VRF satisfies the pseudorandomness property in the random oracle model.

Proof. Let $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ be a PPT adversary against the pseudorandomness of our XM-VRF scheme. Recall that $y_{\text{VRF}} = H_1(\Sigma, x)$ where H_1 is modeled as a random oracle. Also recall that the signature $\Sigma = (\sigma_0, \text{Auth}_0, \sigma_1, \text{Auth}_1, \dots, \sigma_{d-1}, \text{Auth}_{d-1})$ contains WOTS⁺ signatures $\sigma_0, \sigma_1, \dots, \sigma_{d-1}$ which is obtained by iteratively hashing a set of completely independent and uniformly random λ -bit strings unknown to $\mathcal{A}$. So, any σ_i generates a seemingly random bit string, which is an element of the collection Σ . Hence, $\mathcal{A}$ can distinguish y_{VRF} from a uniformly random value only if $\mathcal{A}$ has queried H_1 on the input (Σ, x) . However, this occurs with negligible probability, as $\mathcal{A}$ is unable to query the signing oracle on x . From this point, the pseudorandomness property can be established. We demonstrate this by contradiction, employing the following approach. Suppose the adversary $\mathcal{A}$ wins the experiment $\text{Exp-PRand}_{\mathcal{A}}$ in Algorithm 7 with probability $> \frac{1}{2} + \text{negl}(\lambda)$. So $\mathcal{A}$ can distinguish between λ -bit random string and $H_1(\Sigma, x)$. This implies contradiction to the fact that H_1 is a random oracle. Therefore the adversary can win the experiment $\text{Exp-PRand}_{\mathcal{A}}$ with probability $\leq \frac{1}{2} + \text{negl}(\lambda)$. Hence pseudorandomness holds for our XM-VRF. $\square$

Algorithm 7: Exp-PRand _{$\mathcal{A}$}

1. $\text{pp}_{\text{XM-VRF}} \leftarrow \text{VRF.ParamGen}(1^\lambda)$;
 2. $(\text{sk}_{\text{XM-VRF}}, \text{vk}_{\text{XM-VRF}}) \leftarrow \text{VRF.KeyGen}(\text{pp}_{\text{XM-VRF}})$;
 3. $(x, st) \leftarrow \mathcal{A}_1^{\text{O}_{\text{VRF.Eval}(\cdot)}}(\text{vk}_{\text{XM-VRF}})$;
 4. $(y_0, \pi_0) \leftarrow \text{VRF.Eval}(\text{pp}_{\text{XM-VRF}}, \text{sk}_{\text{XM-VRF}}, x)$;
 5. $y_1 \leftarrow \{0, 1\}^\lambda$;
 6. $b \xleftarrow{\$} \{0, 1\}$;
 7. $y^* \leftarrow \mathcal{A}_2^{\text{O}_{H_1(\cdot)}}(x^*)$;
 8. $b' \leftarrow \mathcal{A}_2^{\text{O}_{\text{VRF.Eval}(\cdot)}}(y_b, st, y^*)$;
-

Theorem 6. XM-VRF satisfies the forward security property.

Proof. XM-VRF employs a stateful forward secure PRG G in their construction, which updates it's internal state after each use. In XM-VRF, the evaluation key is updated using the stateful pseudorandom generator G . Therefore, XM-VRF achieves forward security. $\square$

4 Application of VRF in Blockchain

VRFs serve as a fundamental component in PoS blockchains, facilitating secure and private cryptographic sortition for selecting block proposers and voting

committees [26, 18, 31] and are further employed in smart contract applications, including decentralized online lotteries. VRFs underpin decentralized lotteries and random-beacon schemes, delivering trustworthy, unpredictable randomness that is vital for many blockchain mechanisms notably sharding, where secure and unguessable shard assignments are essential. Dfinity Internet Computer leverages a VRF-based random beacon to generate decentralized randomness, which is subsequently utilized for task assignment, transaction validation, and maintaining network security. For more details look into [30]. In Ethereum 2.0, VRFs are used to securely and randomly assign validators to different shards. Our study emphasizes the role of VRF in enabling cryptographic sortition for blockchain consensus.

Cryptographic sortition, introduced by Algorand, allows users to privately determine their own eligibility to participate in the consensus protocol. In this process, participants remain anonymous to all others including potential adversaries. In Algorand, the committee-based consensus mechanism introduced by Gilad et al. [26] leverages a VRF to perform cryptographic sortition. Moreover, the VRF mechanism in Algorand facilitates a secure and unbiased selection of committee members in a non-interactive manner, where each participant's chance of selection is proportional to their stake. This randomized committee selection in Algorand also mitigates targeted attacks by preventing adversaries from identifying specific committee members in advance. Furthermore, the incorporation of a VRF in Algorand's consensus protocol enhances scalability and performance, enabling the system to efficiently support millions of users. At its core, Algorand employs a Byzantine Agreement protocol [14], run by a randomly chosen committee of users for each consensus round. More precisely, this protocol employs XM-VRF in the following manner:

- i. Every user possesses a secret-public key pair $(sk_{XM-VRF}, pk_{XM-VRF})$.
- ii. Let A denote a block to be appended to the blockchain.
- iii. To decide if they are chosen to join the committee for validating block A , each user follows these procedures:
 - (a) The user computes $(y_{XM-VRF}, \pi_{XM-VRF}) \leftarrow XM-VRF.Eval(sk_{XM-VRF}, Q)$ using their secret key sk_{XM-VRF} and a publicly known randomness value Q and output a pseudorandom value y_{XM-VRF} along with a proof π_{XM-VRF} that the computation was performed correctly.
 - (b) The user then verifies whether y_{XM-VRF} falls within the target range $[0, P]$, where P is a parameter determined by the user's current stake. If this condition is satisfied, the user is selected as a committee member for block A .
- iv. Any user can confirm committee membership by executing the VRF verification algorithm with $pk_{XM-VRF}, Q, y_{XM-VRF}$ and π_{XM-VRF} as input and checking that $y_{XM-VRF} \in [0, P]$.

5 Conclusion

In this work, we propose a straightforward and efficient method to realize VRF functionality tailored for blockchain-based leader election. Our construction ad-

addresses the challenge of long key generation times, enabling a single key pair to produce multiple outputs across different election rounds. This construction can allow a single key pair to provide 2^{80} outputs, an almost unlimited number of outputs virtually. Our approach avoids the race conditions seen in Bitcoin and can be built using standard (post-quantum) cryptographic primitives. Future work can explore the optimizations, aiming to enhance the efficiency and scalability of our approach.

Acknowledgment

We would like to thank the Prime Minister’s Research Fellowship (PMRF), Government of India, for providing financial support for this research. The continued encouragement and resources offered under this fellowship have been instrumental in advancing the progress of this work.

References

1. Abdalla, M., Catalano, D., Fiore, D.: Verifiable random functions from identity-based key encapsulation. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 554–571. Springer (2009)
2. Albrecht, M.R., Rechberger, C., Schneider, T., Tiessen, T., Zohner, M.: Ciphers for mpc and fhe. In: Oswald, E., Fischlin, M. (eds.) Advances in Cryptology – EUROCRYPT 2015. pp. 430–454. Springer Berlin Heidelberg, Berlin, Heidelberg (2015)
3. Alkandari, A.A., Al-Shaikhli, I.F., Alahmad, M.A.: Cryptographic hash function: A high level view. In: 2013 International Conference on Informatics and Creative Multimedia. pp. 128–134 (2013). <https://doi.org/10.1109/ICICM.2013.29>
4. Bellare, M., Yee, B.: Forward-security in private-key cryptography. In: Proceedings of the 2003 RSA Conference on The Cryptographers’ Track. p. 1–18. CT-RSA’03, Springer-Verlag, Berlin, Heidelberg (2003)
5. Bitansky, N.: Verifiable random functions from non-interactive witness-indistinguishable proofs. *Journal of Cryptology* **33**(2), 459–493 (2020)
6. Bodaghi, O., Safavi-Naini, R.: Short paper: Breaking x-vrf, a post-quantum verifiable random function. In: Clark, J., Shi, E. (eds.) Financial Cryptography and Data Security. pp. 90–100. Springer Nature Switzerland, Cham (2025)
7. Boneh, D., Franklin, M.: Identity-based encryption from the weil pairing. In: Annual international cryptology conference. pp. 213–229. Springer (2001)
8. Boneh, D., Gentry, C., Lynn, B., Shacham, H.: Aggregate and verifiably encrypted signatures from bilinear maps. In: Advances in Cryptology—EUROCRYPT 2003: International Conference on the Theory and Applications of Cryptographic Techniques, Warsaw, Poland, May 4–8, 2003 Proceedings 22. pp. 416–432. Springer (2003)
9. Boneh, D., Lynn, B., Shacham, H.: Short signatures from the weil pairing. In: International conference on the theory and application of cryptology and information security. pp. 514–532. Springer (2001)
10. Brakerski, Z., Goldwasser, S., Rothblum, G.N., Vaikuntanathan, V.: Weak verifiable random functions. In: Theory of Cryptography: 6th Theory of Cryptography Conference, TCC 2009, San Francisco, CA, USA, March 15–17, 2009. Proceedings 6. pp. 558–576. Springer (2009)

11. Buchmann, J., Dahmen, E., Hülsing, A.: Xmss-a practical forward secure signature scheme based on minimal security assumptions. In: Post-Quantum Cryptography: 4th International Workshop, PQCrypto 2011, Taipei, Taiwan, November 29–December 2, 2011. Proceedings 4. pp. 117–129. Springer (2011)
12. Buchmann, J., Dahmen, E., Schneider, M.: Merkle tree traversal revisited. In: Post-Quantum Cryptography: Second International Workshop, PQCrypto 2008 Cincinnati, OH, USA, October 17–19, 2008 Proceedings 2. pp. 63–78. Springer (2008)
13. Buser, M., Dowsley, R., Esgin, M.F., Kasra Kermanshahi, S., Kuchta, V., Liu, J.K., Phan, R.C.W., Zhang, Z.: Post-quantum verifiable random function from symmetric primitives in pos blockchain. In: European Symposium on Research in Computer Security. pp. 25–45. Springer (2022)
14. Castro, M., Liskov, B.: Practical byzantine fault tolerance. In: Proceedings of the Third Symposium on Operating Systems Design and Implementation. p. 173–186. OSDI '99, USENIX Association, USA (1999)
15. Chase, M., Lysyanskaya, A.: Simulatable vrfs with applications to multi-theorem nizk. In: Annual International Cryptology Conference. pp. 303–322. Springer (2007)
16. Chen, J., Gorbunov, S., Micali, S., Vlachos, G.: Algorand agreement: Super fast and partition resilient byzantine agreement. Cryptology ePrint Archive (2018)
17. David, B., Gaži, P., Kiayias, A., Russell, A.: Ouroboros praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain. In: Nielsen, J.B., Rijmen, V. (eds.) Advances in Cryptology – EUROCRYPT 2018. pp. 66–98. Springer International Publishing, Cham (2018)
18. David, B., Gaži, P., Kiayias, A., Russell, A.: Ouroboros praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain. In: Advances in Cryptology–EUROCRYPT 2018: 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29–May 3, 2018 Proceedings, Part II 37. pp. 66–98. Springer (2018)
19. Dodis, Y.: Efficient construction of (distributed) verifiable random functions. In: Public Key Cryptography—PKC 2003: 6th International Workshop on Practice and Theory in Public Key Cryptography Miami, FL, USA, January 6–8, 2003 Proceedings 6. pp. 1–17. Springer (2002)
20. Dodis, Y., Yampolskiy, A.: A verifiable random function with short proofs and keys. In: International Workshop on Public Key Cryptography. pp. 416–431. Springer (2005)
21. Esgin, M.F., Ersoy, O., Kuchta, V., Loss, J., Sakzad, A., Steinfeld, R., Yang, X., Zhao, R.K.: A new look at blockchain leader election: Simple, efficient, sustainable and post-quantum. In: Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security. pp. 623–637 (2023)
22. Esgin, M.F., Kuchta, V., Sakzad, A., Steinfeld, R., Zhang, Z., Sun, S., Chu, S.: Practical post-quantum few-time verifiable random function with applications to algorand. In: International Conference on Financial Cryptography and Data Security. pp. 560–578. Springer (2021)
23. Esgin, M.F., Steinfeld, R., Liu, D., Ruj, S.: Efficient hybrid exact/relaxed lattice proofs and applications to rounding and vrfs. In: Annual International Cryptology Conference. pp. 484–517. Springer (2023)
24. Fuchsbaauer, G.: Constrained verifiable random functions. In: Security and Cryptography for Networks: 9th International Conference, SCN 2014, Amalfi, Italy, September 3–5, 2014. Proceedings 9. pp. 95–114. Springer (2014)
25. Gilad, Y., Hemo, R., Micali, S., Vlachos, G., Zeldovich, N.: Algorand: Scaling byzantine agreements for cryptocurrencies. In: Proceedings of the 26th Symposium

- on Operating Systems Principles. p. 51–68. SOSP '17, Association for Computing Machinery, New York, NY, USA (2017). <https://doi.org/10.1145/3132747.3132757>, <https://doi.org/10.1145/3132747.3132757>
26. Gilad, Y., Hemo, R., Micali, S., Vlachos, G., Zeldovich, N.: Algorand: Scaling byzantine agreements for cryptocurrencies. In: Proceedings of the 26th symposium on operating systems principles. pp. 51–68 (2017)
 27. Goldberg, S., Naor, M., Papadopoulos, D., Reyzin, L., Vasant, S., Ziv, A.: Nsec5: Provably preventing dnssec zone enumeration (2015)
 28. Goldwasser, S., Micali, S., Rivest, R.L.: A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on computing* **17**(2), 281–308 (1988)
 29. Goyal, R., Hohenberger, S., Koppula, V., Waters, B.: A generic approach to constructing and proving verifiable random functions. In: Theory of Cryptography: 15th International Conference, TCC 2017, Baltimore, MD, USA, November 12–15, 2017, Proceedings, Part II 15. pp. 537–566. Springer (2017)
 30. Hanke, T., Movahedi, M., Williams, D.: Dfinity technology overview series, consensus system. arXiv preprint arXiv:1805.04548 (2018)
 31. Hellebrandt, L., Homoliak, I., Malinka, K., Hanáček, P.: Increasing trust in tor node list using blockchain. In: 2019 IEEE International Conference on Blockchain and Cryptocurrency (ICBC). pp. 29–32. IEEE (2019)
 32. Hohenberger, S., Waters, B.: Constructing verifiable random functions with large input spaces. In: Advances in Cryptology—EUROCRYPT 2010: 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques, French Riviera, May 30–June 3, 2010. Proceedings 29. pp. 656–672. Springer (2010)
 33. Hülsing, A.: W-ots+—shorter signatures for hash-based signature schemes. In: Progress in Cryptology—AFRICACRYPT 2013: 6th International Conference on Cryptology in Africa, Cairo, Egypt, June 22–24, 2013. Proceedings 6. pp. 173–188. Springer (2013)
 34. Jager, T.: Verifiable random functions from weaker assumptions. In: Theory of Cryptography: 12th Theory of Cryptography Conference, TCC 2015, Warsaw, Poland, March 23–25, 2015, Proceedings, Part II 12. pp. 121–143. Springer (2015)
 35. Kaliski, B.: Pseudorandom Function, pp. 995–995. Springer US, Boston, MA (2011). https://doi.org/10.1007/978-1-4419-5906-5_427
 36. Katz, J., Kolesnikov, V., Wang, X.: Improved non-interactive zero knowledge with applications to post-quantum signatures. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. p. 525–537. CCS '18, Association for Computing Machinery, New York, NY, USA (2018). <https://doi.org/10.1145/3243734.3243805>, <https://doi.org/10.1145/3243734.3243805>
 37. Lai, Y.F.: Capybara and tsubaki: verifiable random functions from group actions and isogenies. *Cryptology ePrint Archive* (2023)
 38. Leroux, A.: Verifiable random function from the deuring correspondence and higher dimensional isogenies (2023)
 39. Lysyanskaya, A.: Unique signatures and verifiable random functions from the dh-ddh separation. In: Advances in Cryptology—CRYPTO 2002: 22nd Annual International Cryptology Conference Santa Barbara, California, USA, August 18–22, 2002 Proceedings 22. pp. 597–612. Springer (2002)
 40. Merkle, R.C.: A digital signature based on a conventional encryption function. In: A Conference on the Theory and Applications of Cryptographic Techniques on Advances in Cryptology. p. 369–378. CRYPTO '87, Springer-Verlag, Berlin, Heidelberg (1987)

41. Micali, S., Rabin, M., Vadhan, S.: Verifiable random functions. In: 40th annual symposium on foundations of computer science (cat. No. 99CB37039). pp. 120–130. IEEE (1999)
42. Papadopoulos, D., Wessels, D., Huque, S., Naor, M., Včelák, J., Reyzin, L., Goldberg, S.: Making nsec5 practical for dnssec. Cryptology ePrint Archive (2017)
43. Proelss, J., Sévigny, S., Schweizer, D.: Gamefi: The perfect symbiosis of blockchain, tokens, defi, and nfts? International Review of Financial Analysis **90**, 102916 (2023). <https://doi.org/https://doi.org/10.1016/j.irfa.2023.102916>, <https://www.sciencedirect.com/science/article/pii/S1057521923004325>
44. Robshaw, M.J.B.: One-Way Function, pp. 887–888. Springer US, Boston, MA (2011). https://doi.org/10.1007/978-1-4419-5906-5_467
45. Yang, R., Au, M.H., Zhang, Z., Xu, Q., Yu, Z., Whyte, W.: Efficient lattice-based zero-knowledge arguments with standard soundness: construction and applications. In: Advances in Cryptology–CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part I 39. pp. 147–175. Springer (2019)

Appendix

A Signature Scheme

Definition 2 (Digital Signature Scheme). [28] *A digital signature scheme corresponding to a given message space $\mathcal{M}$ comprises a suite of probabilistic polynomial-time (PPT) algorithms, represented as $\text{DSS} = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Vrfy})$, with the following requirements:*

$\text{Setup}(1^\lambda) \rightarrow \text{pp}$: *A trusted party taking security parameter λ as input, outputs a public parameter pp and makes it publicly available.*

$\text{KeyGen}(\text{pp}) \rightarrow (\text{sk}, \text{pk})$: *On input pp , a user runs this algorithm to generate a secret key and public key pair (sk, pk) for signing and verification respectively. The secret key sk is kept secret to the user while the public key pk is made public.*

$\text{Sign}(\text{pp}, \text{sk}, M) \rightarrow \sigma$: *Taking input pp, sk and a message $M \in \mathcal{M}$, the signer executes this algorithm to generate a signature σ on the message M . The message-signature pair (M, σ) is sent over a public channel.*

$\text{Vrfy}(\text{pp}, \text{pk}, M, \sigma) \rightarrow 1/0$: *Given inputs pp, pk , a message $M \in \mathcal{M}$ and a signature σ , a verifier checks the validity of the signature σ on M . If the signature is valid, it outputs 1, otherwise 0.*

Correctness. *For all $\text{pp} \leftarrow \text{DSS.Setup}(1^\lambda)$, all $(\text{sk}, \text{pk}) \leftarrow \text{DSS.KeyGen}(\text{pp})$ and all signature $\sigma \leftarrow \text{DSS.Sign}(\text{pp}, \text{sk}, M)$, it must satisfy that*

$$\text{DSS.Vrfy}(\text{pp}, \text{pk}, M, \sigma) \rightarrow 1$$

EU-CMA Security: The standard security notion for the digital signature scheme $\text{DSS} = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Vrfy})$ is *existential unforgeability under adaptive chosen message attacks* (EU-CMA).

Definition 3 (EU-CMA). [28] Let $\lambda, t, q \in \mathbb{N}$, $t, q = \text{poly}(\lambda)$. Digital Signature Scheme $\text{Dss} = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Vrfy})$ is EU-CMA secure if

$$\text{Adv}_{\mathcal{A}, \text{Dss}}^{\text{EU-CMA}}(\lambda; t, q) = \max_{\mathcal{A}} \left\{ \Pr \left[\text{Exp}_{\mathcal{A}, \text{Dss}}^{\text{EU-CMA}}(\lambda) = 1 \right] \right\} \leq \text{negl}(\lambda)$$

for all probabilistic adversaries $\mathcal{A}$, running in time $\leq t$ and making at most q queries to $\mathcal{O}_{\text{Sign}(\text{sk}, \cdot)}$ in the experiment $\text{Exp}_{\mathcal{A}, \text{Dss}}^{\text{EU-CMA}}(\lambda)$ between a challenger and an adversary defined in Experiment 1 below for some negligible function $\text{negl}(\cdot)$.

Experiment 1: $\text{Exp}_{\mathcal{A}, \text{Dss}}^{\text{EU-CMA}}(\lambda)$

Setup: The challenger takes input λ to generate public parameter $\text{pp} \leftarrow \text{Dss.Setup}(1^\lambda)$. Then challenger produces key pairs $(\text{sk}, \text{pk}) \leftarrow \text{Dss.KeyGen}(\text{pp})$. It maintains a list Qlist and initializes $\text{Qlist} = \emptyset$. It gives the public parameter pp and pk to adversary $\mathcal{A}$ and keep sk secret to itself.

Query Phase: The adversary $\mathcal{A}$ repeats polynomial times sign query on messages of its choice to obtain signatures from the challenger. For a signature query on a message $M \in \mathcal{M}$ from $\mathcal{A}$, the challenger generates signature $\sigma \leftarrow \mathcal{O}_{\text{Sign}(\text{sk}, \cdot)}(M)$ and send it to $\mathcal{A}$ and update $\text{Qlist} = \text{Qlist} \cup \{M\}$.

Forgery: The challenger in turn checks whether adversary $\mathcal{A}$ submits a forgery (M^*, σ^*) to $(\text{Dss.Vrfy}(\text{pp}, \text{pk}, M^*, \sigma^*) \rightarrow 1 \text{ and } M^* \notin \text{Qlist})$. If verification succeeds, then return 1, otherwise return 0.

Definition 4 (Unique Signature Scheme). [36] Let $\text{Dss} = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Vrfy})$ be a signature scheme and $\Sigma(m, \text{pk}) = \{\sigma : \text{Dss.Vrfy}(\text{pp}, \text{pk}, m, \sigma) = 1\}$. Then Dss is said to be a unique signature if $|\Sigma(m, \text{pk})| = 1$ for all m and pk .

B Function Family

Definition 5 (Second Preimage Resistant). [3] A function $f : A \rightarrow B$ is said to be second preimage resistant if given $x \in A$ there exist no polynomial time adversary which can find another $x' \in A$ such that $f(x) = f(x') \in B$. If each $f \in \mathcal{F}$ is second preimage resistant for a function family $\mathcal{F}$, then $\mathcal{F}$ is second preimage resistant (SPR).

Definition 6 (One-way). [44] A function f is a one-way function if the following holds

- i. The description of $f : A \rightarrow B$ is publicly known and does not require any secret information for its operation.
- ii. Given $x \in A$, it is easy to compute $f(x) \in B$.
- iii. Given $y \in B$, it is hard to find an $x \in A$ such that $f(x) = y \in B$. More precisely, any polynomial time algorithm succeeds in inverting f with negligible probability.

If each $f \in \mathcal{F}$ is one-way for a function family $\mathcal{F}$, then $\mathcal{F}$ is one-way (OW).

Definition 7 (Undetectable). [35] *A function $f : A \rightarrow B$ is undetectable if there is no polynomial time distinguisher which can distinguish between $f(x) \in B$ and $r \xleftarrow{\$} B$. If each $f \in \mathcal{F}$ is undetectable for a function family $\mathcal{F}$, then $\mathcal{F}$ is undetectable (UD).*

C Stateful Forward Secure PRG

Bellare et al. [4] proposed stateful pseudo-random generators (PRG) where the maximum number of outputs that the PRG is permitted to generate is considered as a parameter of this stateful PRG. Given a state, it outputs a new state and some output bits.

Definition 8 (Stateful PRG). [4] *A stateful PRG G with security parameter λ is mapping from $\{0, 1\}^\lambda$ to $\{0, 1\}^{2\lambda}$ which takes an initial state $s \in \{0, 1\}^\lambda$ as input and generates a new state $s' \in \{0, 1\}^\lambda$ and $r \in \{0, 1\}^\lambda$ as output. And Q is the maximum number of outputs the PRG is allowed to produce.*

A stateful PRG G must satisfy the following properties:

Pseudorandomness: Let $\mathcal{A}$ be a polynomial-time adversary playing the experiment $\text{Exp}_{\mathcal{A}, \text{PRG}}^{\text{PRand}}(\lambda)$ in **Experiment 2**.

Experiment 2: $\text{Exp}_{\mathcal{A}, \text{PRG}}^{\text{PRand}}(\lambda)$

Setup: The challenger first choose $s \in \{0, 1\}^\lambda$ and $b \in \{0, 1\}$ uniformly random.

Query Phase: The adversary $\mathcal{A}$ repeats polynomial times query to oracle $\mathcal{O}^G(s)$ to obtain random λ -bit from the challenger. For all queries oracle $\mathcal{O}^G(s)$ uses either $r_0 \in \{0, 1\}^\lambda$ where $G(s) = (s', r_0)$ or randomly chosen $r_1 \in \{0, 1\}^\lambda$ to respond $r_b \in \{0, 1\}^\lambda$ to the adversary $\mathcal{A}$. So, all queries to $\mathcal{O}^G(s)$ should be indistinguishable from random.

Response: The adversary $\mathcal{A}$ submits a response $b' \in \{0, 1\}$ for that challenge. If $b = b'$, then return 1, otherwise return 0.

A stateful PRG G is said to satisfy pseudorandomness if

$$\text{Adv}_{\mathcal{A}, \text{PRG}}^{\text{PRand}}(\lambda) = \max_{\mathcal{A}} \left\{ \Pr \left[\text{Exp}_{\mathcal{A}, \text{PRG}}^{\text{PRand}}(\lambda) = 1 \right] \right\} \leq \frac{1}{2} + \text{negl}(\lambda)$$

Forward Security: Let $\mathcal{A}$ be a polynomial-time adversary playing the following experiment $\text{Exp}_{\mathcal{A}, \text{PRG}}^{\text{Forward}}(\lambda)$ in **Experiment 3**:

Experiment 3: $\text{Exp}_{\mathcal{A}, \text{PRG}}^{\text{Forward}}(\lambda)$

Setup: The challenger first choose $s \in \{0, 1\}^\lambda$ and $b \in \{0, 1\}$ uniformly random.

Query Phase: The adversary $\mathcal{A}$ repeats polynomial times query to oracle $\mathcal{O}^G(s)$ to obtain random λ -bit from the challenger. For all queries oracle $\mathcal{O}^G(s)$

uses either $r_0 \in \{0,1\}^\lambda$ where $G(s) = (s', r_0)$ or randomly chosen $r_1 \in \{0,1\}^\lambda$ to respond $r_b \in \{0,1\}^\lambda$ to the adversary $\mathcal{A}$. So, all queries to $\mathcal{O}^G(s)$ should be indistinguishable from random. Adversary $\mathcal{A}$ has access to an extra oracle $\mathcal{O}^{\text{State}}()$ to obtain the current state s .

Response: The adversary $\mathcal{A}$ submits a response $b' \in \{0,1\}$ for that challenge. If $b = b'$, then return 1, otherwise return 0.

A stateful PRG G is said to satisfy (t, q, ϵ) -forward security if for any adversary $\mathcal{A}$ running in time atmost t , making atmost q queries to $\mathcal{O}^G(\cdot)$ and one call to $\mathcal{O}^{\text{State}}$

$$\text{Adv}_{\mathcal{A}, \text{PRG}}^{\text{Forward}}(\lambda) = \max_{\mathcal{A}} \left\{ \Pr \left[\text{Exp}_{\mathcal{A}, \text{PRG}}^{\text{Forward}}(\lambda) = 1 \right] \right\} \leq \frac{1}{2} + \epsilon$$